Noname manuscript No.
(will be inserted by the editor)

Assessing Open Source Software Health in Organizations' Intake Processes: A Qualitative Study on the Practitioners' Perspective

Johan Linåker · Thomas Olsson · Efi Papatheocharous

Received: date / Accepted: date

Abstract Context: The increasing reliance on Open Source Software (OSS) in organizations' software supply chains necessitates robust mechanisms in the intake process to ensure sourced components' long-term viability and maintenance. Assessing OSS project health in the intake process is complex due to the wide range of socio-technical factors involved. **Objective**: This study aims to explore how the health of OSS projects may be assessed by practitioners from organizations' intake perspective. **Method**: We conducted a qualitative interview survey with 17 industry experts to identify aspects and related metrics of OSS health. These were mapped against literature and two existing industry frameworks. A subset was identified and applied through a case study at a large international automotive manufacturer. **Results**: 21 health aspects with 72 connected metrics were identified covering community productivity and stability, orchestration, production processes, and outputs. Many metrics map against industry frameworks, while qualitative aspects are missing support. Special consideration is needed when assessing and comparing the health of OSS projects, including their life-cycle stage, complexity, governance concentration, and strategic importance for the focal organization. The case study shows that not all aspects and metrics may be leveraged due to resource constraints and complexity. Instead, subsets of metrics need to be prioritized, and applied in a structured approach using qualitative and quantitative means. **Conclusions**: The study provides a foundation and a starting point for developers in introducing health assessments of OSS components in their intake processes, while also pushing convergence towards a common corpus of health assessment in practice.

J. Linåker
RISE Research Institutes of Sweden, Sweden. E-mail: johan.linaker@ri.se

T. Olsson
Robert Bosch AB, Sweden. E-mail: thomas.olsson@se.bosch.com

E. Papatheocharous
RISE Research Institutes of Sweden, Sweden. E-mail: efi.papatheocharous@ri.se

Keywords Open Source Software, Software Ecosystem, Health, Sustainability, Software Quality, Software Intake.

1 Introduction

The presence and significance of Open Source Software (OSS) in organizations' software supply chains, products, and services are indisputable and ever-growing [26, 35, 87]. By extension, so is the dependence on the OSS project's ability to stay viable and maintained long-term without interruption or weakening, also referred to as the OSS project health [29, 47]. Threats to the health of an OSS project may come in many shapes and forms. Toxicity [89] and non-inclusive culture [71], non-responsive communication [84], and lack of documentation [8] may prevent the on-boarding of new contributors. Burnout or change of interest may cause maintainers to shift away from the OSS project [55]. Projects maintained by only a few individuals may risk becoming abandoned by consequence or have bugs and vulnerabilities emerge [87].

In earlier review work, we show how OSS project health is a broad topic, identifying 107 health aspects ranging across the socio-technical spectra of the OSS communities' peer-production processes, its orchestration and governance, and the software and other deliverables coming out of production process [47]. The health aspects could further be viewed from a project-centric or on a network level of abstraction within their respective ecosystems[1]

The review further shows several limitations in extant research, including tools and models, focused on OSS health assessment. Previous work [86] either outdate the recent rise of health and sustainability research [47], or is commonly limited to either the ecosystem (considering both [6, 97] or project level [44, 82] of analysis, quantitative [63, 68] over qualitative metrics, or focused on designing automated tooling before deriving empirically grounded metrics [28, 31]. Notable exceptions rise from practice in the CHAOSS (Community Health Analytics Open Source Software)[2] and OpenSSF (Open Source Security Foundation) ScoreCard[3] communities, respectively, which have developed broad sets of metrics through open discussions and co-design.

In this study, we aim to bridge and complement the two latter frameworks through empirical research, highlighting what aspects are pivotal to consider when assessing the health of an OSS project from the practitioners' perspective. Specifically, we ask the question of *how the health of OSS projects may be assessed by practitioners from an organizational intake perspective*, where the engineers are regularly faced with the process of identifying and adopting OSS components for integration into their organizations' products, tools

[1] With ecosystem we refer to software ecosystems [51] where software components (OSS projects) can be connected through dependency relationships (as in supply chains), but also related through the functional context they share (e.g. technology stack or programming language). The term supply chain is used when there is a reference is towards cases with explicit dependencies.

[2] `https://chaoss.community/`

[3] `https://securityscorecards.dev/#the-checks`

and infrastructure. This perspective has received limited attention with some notable exceptions, as that of Poth et al. [68], although their focus is on developing tool-support for the quantitative evaluation and less on qualitative metrics, where we consider health from a wider, holistic point of view [47].

We answer the question in three steps. As a *first* step, a qualitative survey [52] based on semi-structured interviews of 17 experts from the industry and OSS ecosystem on what aspects they consider important and how these aspects can be further characterized with relevant metrics from their experience and point of view. Five areas of health aspects that need consideration are identified, including the OSS community's productivity and stability, its orchestration, and the production process and output. Across the five areas, 21 health aspects are identified along with a total of 72 metrics that further help to characterize the different health aspects.

In a *second* step, a mapping was created between health aspects and related metrics from interviews with literature as well as the two industry frameworks of CHAOSS and OpenSSF ScoreCard. The mapping against industry frameworks shows a convergence towards a common corpus for how OSS health may be assessed by practitioners. Still some discrepancies stand out, warranting further research in to how OSS health is assessed, specifically from the organizational intake-process. While there are some blind spots, supporting evidence for the derived metrics can be found across literature.

In the *third* step, a case study [75] was performed to further explore how the health of OSS projects can be assessed in practice from organizations' intake perspective. The case study, performed at a large international automotive manufacturer with high OSS dependencies present, shows that the health assessment is indeed a complex task. Considering all aspects and related metrics would incur both high costs and friction to the development process beyond what is considered acceptable. Instead, health assessments in practice need to balance between managing the risk an OSS dependency can bring, with the cost that the assessments bring. The case study illustrates how health aspects and related metrics can be narrowed down, and implemented through a questionnaire that developers can apply to OSS projects of internal interest. A candidate process is designed and proposed based on the context and needs of the case company.

Our study makes the following **contributions**:

- An empirically grounded mapping of how OSS health can be assessed from the practitioners' perspective in an organizational intake process.
- Five key areas of OSS health: community productivity and stability, orchestration, production processes, and outputs, incorporating 21 health aspects with 72 connected metrics (see Fig. 2 and Appendix A).
- A mapping between identified metrics against extant research and two established industry frameworks, highlighting gaps in qualitative aspects and complementary nature of the frameworks, while validating of metrics identified in study and pushing convergence towards a common corpus of health assessment in practice.

- A characterization of four OSS project traits, including life-cycle stage, complexity, governance concentration, and strategic importance, and how they influence the way in which the health of OSS projects should be assessed and compared.
- An example process and implementation of health assessment in the context of an organizational intake process through a case study at a large international automotive manufacturer (see Fig 3).
- A foundation for future research to take point from in the continued exploration of how OSS health can be assessed, compared, and contrasted considering contextual aspects of the OSS projects.

The rest of the paper is structured as follows. In Section 2, we discuss related work on OSS quality models and health assessment. Our research design spanning over two cycles is presented in further detail in Section 3. The health aspects and related metrics identified in are presented in Section 4, followed by the presentation of health assessment may be applied in practice through our case study in Section 5. In Section 6.2, we discuss project traits that may impact how OSS projects should be assessed and compared. This is followed in Section 6 by a discussion on how our contrast with related work, and its limitations and how threats to validity are managed. Finally, the main conclusions of the study are summarized and presented in Section 7, followed by Appendixes for the 72 metrics, and related data.

2 Background and Related work

Below, we first provide an overview of related work on OSS health assessment based on our earlier literature survey of 146 publications [47]. Second, we explain and motivate the scope of our study, and how it addresses a gap in research contrasted through the related work.

2.1 OSS health assessment

In our earlier work [47], we surveyed 146 papers through a snowball literature review [100], identifying 107 health aspects ranging across the socio-technical spectra of the OSS communities' peer-production processes, its orchestration and governance, and the software and other deliverables coming out of production process [47]. The health aspects could further be viewed from a project-centric or on on a network level of abstraction within their respective ecosystems [51]. The health aspects are divided across 15 themes as illustrated in table 2 in Appendix B as presented in our earlier work [47].

Various processes and methods has been proposed through the years attempting to synthesize parts of these health aspects. One stream of research focus on the evaluation of OSS projects' quality aspects [86], which may generally be described as quantitative means of aggregating technical metrics to arrive at a sourcing decision [63, 77, 88, 99]. Petrinja and Succi, e.g., propose

the QualiPSo Open Source Maturity Model (OMM) based on the more general Capability Maturity Model Integration (CMMI) [63]. They focus specifically on analyzing the development process and evaluating the OMM through desk research on six OSS projects.

Some works focus less on metric definition and specifically on tool development and automation. Goeminne and Mens developed a tool for analyzing the developer activity across OSS projects within a wider ecosystem [28]. The tool is evaluated through an empirical case study of the GNOME OSS ecosystem. In an underpinning and earlier work, authors developed an automated tool for visualization and analysis of OSS project health leveraging metrics identified through the FLOSSMetrics.org project [27]. Samoladas et al. selected a set of common source code metrics through literature and developed the SQO-OSS quality model and an automated analysis tool for applying the model to OSS projects [77]. In more recent work, Gonzalez-Barahona et al. present a new generation of toolsets emerging that supports software development analytics, OSS included from the perspectives of companies, developers, and OSS foundations [31].

Certain works have a more explicit focus on the empirical investigation of OSS projects and their ecosystems, using OSS health as a lens of analysis. Kabbedijk and Jansen, e.g., perform an empirical investigation of the Ruby ecosystem and its large set of OSS projects (gems) [38]. They use a set of code and social media centrality metrics to analyze the activity and collaboration across the projects. Ververs et al. empirically investigate the Debian community to identify factors that promote developer participation in the development [93]. Gamalielsson et al. quantitatively investigate the Nagios OSS project's health using social network centrality metrics [23]. They look specifically at both individuals and service providers. Oriol et al. developed a tool, OSS-CARE [59], that implements a number of key health indicators from the Queso quality model [22], such as activeness, by aggregating individual metrics, such as a number of contributors and open bugs. The tool focuses specifically on an ecosystem level, considering multiple OSS projects integrated, further illustrated by their evaluation of the Eclipse Foundation's rich set of OSS projects.

Several works have taken an ecosystem perspective on health, stemming from the natural ecosystem analogy. Wang et al., e.g., designed an algorithm using a natural ecosystem analogy and developing related indicators through a grounded theory investigation of literature [97]. Carvalho developed a framework for health evaluation of software ecosystems (not specific for OSS) [6]. Fifty-eight metrics were collected from four previous studies and operationalized through automated tool-support. The framework is evaluated through desk research on a scientific software ecosystem. Van Lingen et al. present a framework with health indicators based on related work to evaluate the health of three Content management software ecosystems [92]. The indicators are evaluated using either computation, manual inspection, or a survey of community participants.

There has also been work focused on developing metrics and more or less comprehensive support on the project level. Liao et al. propose a model for predicting the OSS project health using GitHub data [44]. Valiev et al. empirically investigated a series of metrics through a mixed-methods investigation of the PyPi OSS ecosystem [91]. Qiu et al. [70] elicited health indicators by surveying maintainers and operationalized the indicators in a dashboard. The dashboard is focused on displaying social aspects of the community development efforts, thereby raising awareness and providing guidance for the maintainers to improve their health accordingly. Guzani et al. also developed a dashboard aimed at maintainers presenting metrics for improving attraction and retention of newcomers [34]. Singh et al. [82] focus on the community aspects, developing an assessment framework for ranking and comparing communities using the "Order of Preference by Similarity to Ideal Solution from Multi-Criteria Decision-Making toolkit." The framework is developed based on GitHub data and validated through the application on nine OSS projects, which are compared and contrasted in terms of their community health. Poth et al. [68] developed an automated tool-support labeled Open Source Quality Radar (OSQR) to be used by development teams within the Volkswagen Group IT for self-selection of OSS components. The tools extract data from GitHub based on a set of metrics related to community, code quality, and issue management, which was derived from internal experts.

Extant research has further taken several perspectives in the development and application of OSS health metrics. Shaikh and Levina investigate the potential for growing and building alliances and business relations through OSS communities [80]. Li et al. [43], Adewumi et al. [1], and Spinelli [83] in turn focus on factors to consider before sourcing a specific OSS component, similar to Poth et al. [68]. Butler et al. look at factors that can improve internal capabilities to consume OSS [3]. Guizani et al. [34] and Qiu et al. [70] take the maintainers' perspective on how they can monitor and improve the health of their projects.

Beyond research, practices have also evolved through communities such as CHAOSS (Community Health Analytics Open Source Software)[4] and OpenSSF (Open Source Security Foundation)[5]. CHAOSS specifically has developed health metrics using a goal-question-metric model, typically through joint discussions between practitioners, researchers, and community participants. The metrics are themed under different focus areas, each with a specific goal that the metrics aim to provide answers to, e.g., to *"[i]dentify organizations and individuals who make contributions (e.g., contribution type, demographics)"* under the focus area of People[6]. No instructions are provided on which metrics to choose, which is left for the user to decide. OpenSSF has developed a Scorecard[7] with

[4] `https://chaoss.community/`

[5] `https://openssf.org/`

[6] `https://github.com/chaoss/wg-metrics-development/tree/main/focus-areas/people`

[7] `https://github.com/ossf/scorecard`

a subset of quality metrics implemented in a tool that can scan and evaluate repositories accordingly.

2.2 Research gap

In contrast to related work, our study and contribution stand out in several ways. For example, our proposed health aspects and related metrics are not limited to considering either the ecosystem [6, 97] or project level [44, 82] of analysis, quantitative [63,68] over qualitative metrics [47], or focused on designing automated tooling before deriving empirically grounded metrics [28, 31]. The empirical investigation by Van Lingen represents an exception but is, however, more focused on investigating the problem context and less on designing a solution proposal [92]. The notable work by Qiu et al. [70] and Guizani et al. [34] provides an empirically grounded dashboard and support for health monitoring. These are, however, developed primarily from the maintainers' perspective, while our focus is on the organizational intake and dependency monitoring perspective. Further, a substantive part of the current body of research on evaluation models [86] outdate the recent rise of health and sustainability research, which arguably involves far more potential aspects and metrics that were not originally considered [47].

Poth et al. developed a tool for quantitatively analyzing the health of OSS projects for the internal teams of Volkswagen Group IT to use when sourcing OSS components [68]. While taking the organizational intake perspective similar to our study (and others [43, 83]), they focus primarily on developing tool-support for the quantitative evaluation and less on qualitative metrics. Our study further considers the additional use case of enabling the analysis of existing OSS dependencies, also from the organizational perspective. An additional general limitation among related work is that any empirical validation of proposed models or tools beyond desktop research of smaller samples of OSS projects is limited. In our study, we perform a case study at a large international automotive manufacturer and consumer of OSS to evaluate and demonstrate the applicability of our proposed health aspects and metrics.

Closest to our work is that of practice in the CHAOSS and OpenSSF communities. While their work comes out through discussion and co-design, our work comes out of an empirical research process founded in a thorough review of scientific literature, refined through a rigorous qualitative survey, and later applied and evaluated through a real-world case study. Further, CHAOSS presents metrics that can be considered but leaves the user to self-select which to implement and how. Through our case study, we exemplify how practitioners can go about when implementing a health assessment process within an organization's OSS intake process based on a case study. Our contributions, therefore, provide input to the respective communities, on the one hand, with scientific grounding and input to their extant and future metrics, and on the other, with real-world examples for how their metrics can be applied in practice.

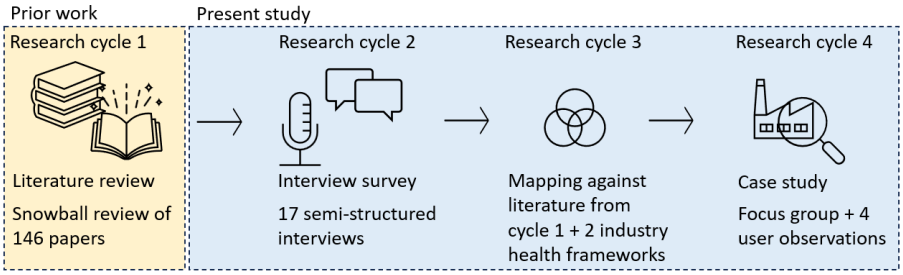

Fig. 1 Overview of the research process spanning over three research cycles. Building on prior work from cycle 1 [47], this study presents the outputs from cycles 2, 3, and 4, consisting of a qualitative interview survey, mapping against literature [47] and two industry frameworks, and a case study, respectively.

A final important difference and contribution regards our characterization of how OSS projects need to be assessed and compared in different ways depending on contextual factors, including their life-cycle stage, complexity, governance concentration, and strategic importance. Models and frameworks from extant research [47] provide little to no consideration of how different projects can be analyzed and compared. The four traits identified and characterized in this study, hence, provide an important frame for practice when implementing a health assessment process and for future research in exploring relationships between OSS health metrics across projects with different traits.

3 Research Design

This study is part of an overarching research project spanning over four research cycles, as visualized in Fig. 1. The first cycle is constituted by our prior research where we explored the literature to find out how the health of an OSS project may be assessed [47]. In this study, we continue the research process through a second, third and fourth research cycle, and explore **how the health of an OSS project can be assessed from practitioners' perspective in the context of an organizational intake process**, where risk and resources available need to be balanced.

We started with an exploratory qualitative survey [52] based on semi-structured interviews with experienced practitioners (research cycle 2, see Fig. 1) where we narrowed down five areas with a total of 21 health **aspects** with 72 related metrics (see Appendix A) considered critical by practitioners, each covering *a particular part of OSS project health that can cause issues with consequences for the OSS project, its community, and end-users*. In a third research cycle, health aspects and metrics were mapped against extant research [47] and metrics from two established industry frameworks to validate and gain a more comprehensive understanding of the practitioners' perspective.

Building on our qualitative insights, we extend our exploration in a fourth research cycle through a case study [74] of a real-world context to understand how health assessment can be instantiated in organizational intake process. The process started with a focus group [39] where participants in groups prioritized aspects and related metrics most important in their context and practice. A questionnaire was generated, including the concerned aspects and metrics, and then evaluated through a set of user observations where participants from the focus group individually applied the questionnaire to evaluate the health of the OSS project of their choice. Below, we present the research approach in further detail.

3.1 Qualitative Interview Survey

A qualitative survey design leveraging semi-structured interviews as a means of data collection was, hence, deemed suitable considering the exploratory nature of the study [52].

Interviewees were identified and selected from two groups using a combination of purposive and convenience sampling techniques [52]. The first group focused on a specific case company, which included experts from program management, cybersecurity, tools- and infrastructure, and product engineering. This sampling was motivated by the fact that we wanted to get a comprehensive view of the aspects from a confined context. By sampling interviewees from different parts of a case company, we could get complementary perspectives of aspects that were considered important for the case company at large. Interviewees were identified through snowball recommendation from our first interviewee, the open source program manager, until interviewees and we considered that all relevant views of the company had been captured.

Our second group of interviewees consisted of general experts with 10+ years of professional experience in working with OSS from a community or company perspective, either on a technical or strategic level. All interviewees also confirmed that they had repeated experience in analyzing the health of OSS projects, although based on their personal experience and rationale. Our intention was to improve the generalizability of our qualitative interview survey, although we acknowledge the limitations that come with our qualitative approach and limited overall interview sample. After the fifteenth interview authors 1 and 2 were observing a theoretical saturation in the findings as no new themes or significant codes were emerging through the continuous coding process, confirmed by the following interviews. In total, 17 interviews were conducted, of which five were from the first group sample and 12 from the second (see Table 4).

A semi-structured approach was adopted, where interviewees were asked open questions derived from the higher level dimensions of our earlier reported framework [47] (see Table 2 in Appendix B). Specifically, we asked, following our definition of OSS health:

- How would you characterize a healthy OSS project?

- What aspects would you consider in terms of the community of individuals developing the OSS?
- What aspects would you consider in terms of the OSS and other deliverables produced by these individuals?
- What aspects would you consider in terms of the orchestration, governance, and coordination of the development?
- What aspects would you consider in terms of the OSS project in isolation?
- What aspects would you consider in terms of the OSS project in relation to its dependencies and broader ecosystem?

After being allowed initial open reasoning, the interviewee was asked to reflect specifically on each related theme within the specific dimension that had not yet been touched upon, using definitions as provided in our earlier reported framework (see Table 2 in Appendix B). Each interview lasted for about 60 minutes and was conducted by the first, and at least one of the second and third authors. Interviews were conducted online, recorded, and automatically transcribed. Interviewees were provided with a copy of the transcript with the option to correct, add, or retract any statement.

The transcripts were coded using structural coding [76] separately by both the first and second author using our earlier reported framework derived from literature [47] as an *a priori* code book (see section 7.5). Codes with two or fewer mentions were excluded from the analysis to ensure a minimum level of relevance and generalizability. We recognize that minor low-frequency codes can still reflect important or sensitive practitioner experience, but consider them as a minimum level of validation for the derived findings. After axial coding and selective coding [76], 21 themes, or health aspects, divided in five main areas were synthesized (see Table 1). The a priori and posteriori codebooks, along with the tracing between them, are available in the supplementary material (see section 7.5). Across the health aspects, 72 metrics (see Fig. 2 and Appendix A) were derived from the data using open and axial coding [76] performed by the first author and continuously validated through peer debriefing with the second author.

Four of the codes, originally thought of as health aspects, with time inductively emerged as specific traits that signaled an impact in how the various aspects and metrics should be interpreted and applied when evaluating and comparing projects. The traits include the life-cycle, complexity, and governance concentration of the OSS project and its strategic importance to the organization analyzing the project as part of its intake process. These were validated in the case study through the focus groups discussions as valuable for framing any project in the inception of a health assessment.

3.2 Mapping against Industry Frameworks

To gain a validate and gain further understanding of how the health of an OSS project can be assessed from practitioners' perspective, we map health aspects and related metrics identified in the qualitative survey against two

established industry frameworks for assessing OSS health. The two frameworks, CHAOSS (Community Health Analytics Open Source Software)[8] and OpenSSF (Open Source Security Foundation) Scorecard[9] contains 89 and 18 metrics respectively. For a detailed description of each, the reader is referred to the frameworks' websites.

The first author performed the mapping against the CHAOSS framework. First by familiarizing with the various metrics, and then by cross-checking each of the proposed metrics coming from the interview survey, against the framework metrics. The process was iterated for OpenSSF Scorecard. The mapping was validated with one long-term contributor from each of the framework communities. The mapping could in many cases suggest multiple framework metrics for each of the metrics proposed from the interview survey. 10 and 46 cases of overlap between metrics identified from interview survey and the two frameworks are identified. Accordingly, in 20 instances, no corresponding framework metrics were identified (26 and 62 for CHAOSS and OpenSSF Scorecard respectively) signaling a gap between industry practice and metrics elicited from the interview survey. The mapping is available in the supplementary material to this paper (see Section 7.5).

To further validate the identified metrics from practitioners, these were also mapped against literature identified in our earlier survey of how OSS health can be characterized. Supporting evidence was found for 61 out of the 72 identified metrics, where the 11 missing were distributed across 7 aspects, where Licensing, Security, and Adoption had the highest representation with 2 occurrences each.

3.3 Case study: Health Assessment in a Real-world Context

An exploratory case study approach [75] was adopted to further narrow down what health aspects matters for practitioners in an organizational intake process, considering a real-world context, and any organizational factors and constraints that apply.

The case company is a large international automotive manufacturer with 50,000+ employees and a yearly turnover of 13 billion Euro. They use OSS both on board the vehicles in a safety-critical environment (although limited), in the cloud for enabling connected services, and in the internal development and infrastructure environment. Our knowledge and understanding of the case company has extended throughout a collaborative and longitudinal two year process as they constitute an industrial partner in a two year project, in which this study was conducted. The interviewees from the case sample of the qualitative survey (see table 4) further helped to provide contextual knowledge, and consider the organization's perspective in elicitation of the health aspects and related metrics.

[8] `https://chaoss.community/`

[9] `https://github.com/ossf/scorecard`

3.3.1 Focus Group

A focus group design [39] was adopted, incorporating a dedicated development team with various roles but focused on the same software stack. The team selected included 16 individuals focusing on developing and maintaining the internal tools and infrastructure for software development, testing, and building pipelines within the case company, why they all shared a common focus and frame of reference.

In groups, participants prioritized the top five most important health aspects (as presented in section 4) and motivated why (hand-outs available in the supplementary material, see Section 7.5). In the second step, the groups were asked to prioritize the most important metric per aspect (also as presented in section 4). Finally, all groups discussed their general thoughts about the health assessment process, how it can be implemented into the current development processes, and what the barriers might be. Each group made their own notes, which were collected by the first author. After the group discussions, the whole group openly discussed the top prioritized health aspects and related metrics, concrete examples of where a health assessment process would have been needed in the past, and how such a process could look in practice.

Following the focus group, the first author made further notes summarizing the open discussion of the focus group. The participants' priorities were cross-compiled, resulting in a new and briefer set of health aspects and related metrics than earlier elicited from the qualitative interview survey (see table 3 in Appendix C). A candidate health assessment process (see section 5) was also drafted based on input from the focus group. The list of health aspects and related metrics, as well as the candidate process, was member-checked with participants from the focus group.

3.3.2 User observations

To evaluate the applicability, and understandability of elicited health aspects and related metrics generated as output from the focus group, a set of user observations was planned using a "think-aloud" strategy [41] . The elicited health aspects and related metrics was transformed into a questionnaire (see Section 7.5), which participants from the focus group individually applied to evaluate the health of two OSS projects of their choice. In total, four user observation sessions were performed, with sampling done in collaboration with the manager (I1) of the team who attended the focus group.

Observation sessions lasted between 60-120 minutes and were managed and recorded over an online videoconferencing platform. The first author of this study facilitated the sessions. Each session was initiated with an overview of the research project, the scope and purpose of the session, and the revised questionnaire that came out of the focus group prioritization. Following this, the user provided a background and connection to the OSS project to be evaluated. The user then performed the evaluation based on the provided

questionnaire. The first author, facilitating the evaluation, took notes continuously and kept an observational role throughout the evaluation. After each evaluation, the questionnaire was revised to improve clarity and context.

4 Findings

Below, we present the synthesized output from our qualitative investigation into how the health of an OSS project can be assessed from practitioners' perspective in the context of an organizational intake process (see Fig. 2). Five areas, with 21 aspects, have been identified from the interviews, each aspect covering *a particular part of OSS project health that can cause issues with consequences for the OSS project, its community, and end-users*. For each aspect, we briefly discuss supporting evidence and findings from extant work [47].

For each aspect, a number of **metrics** are defined to help *break down and enable the analysis of a concerned aspect in regards to an OSS project*. The metrics are available in Appendix A where the mapping against literature and the industry frameworks CHAOSS and OpenSSF Scorecard are available, as also indicated in Fig. 2.

- Community productivity - Aspects describing the pulse and activity that contribute to the collaborative development of the OSS project.
- Community stability - Aspects describing the robustness and ability of the community to withstand any interruption or change that may impact its population of maintainers and contributors,
- Orchestration - Aspects related to the governance and coordination mechanisms that enable the development and collaboration within the OSS community.
- Production process - Aspects of the peer-production process generating the OSS project outputs.
- Production outputs - Aspects describing the quality and comprehensiveness of the OSS project outputs, including source code and documentation.

See table 1 for an overview of the concerned areas, aspects, and metrics.

Table 1: Overview of the health aspects elicited from the qualitative interview survey.

Aspect	Description
Community productivity	
Social activity	The activity from the OSS project's community and maintainers both in online channels and physically.
Responsiveness	The time to a response from the maintainers and community towards, e.g., discussions, pull requests, or issues.
External visibility	Visibility of the OSS project to an audience outside the community of individuals actively engaged in the project.

Development activity	The overall development activity by the community, including the many technical aspects and deliverables of the OSS project.
Development efficiency	The effectiveness and ease of the maintainers and contributors of an OSS project in managing and moving the development forward.
Community stability	
Adoption	Usage and technical adoption of the OSS project as a dependency in downstream software projects and by end-users.
Organizational diversity	The diversity of organizations within an OSS community in terms of governance, contribution, and adoption of the underpinning project.
Demographical diversity	The individual level of the maintainer and contributors to an OSS project in gender, race, time zone, language, and cultural aspects.
Discussion climate	The discussion climate in the community in regard to sentiment, tone, and manner in answers, messages, and general communication within the OSS project, as well as how potential conflicts are managed.
Knowledge concentration	The concentration or distribution of contributions and knowledge to specific individuals or groupings within an OSS project (also referred to as the bus or truck factor).
Contributor turnover	The attraction, retention, and attrition of maintainers and contributors to an OSS project.
Financial sustainability	The financial situation of maintainers and contributors of OSS projects and whether it enables sustainable and dedicated time for maintenance of the projects.
Orchestration	
Governance structure	The explicitness, formality, and general recognition of the ecosystem's governance structure and leadership.
Openness	To what extent the OSS project is welcoming to and accepting contributions and considering new ideas and general input and influence on the project's development from existing and new contributors.
Licenses	License-related aspects and processes of managing and distributing the intellectual property maintained by the OSS project.
Production process	
Development process	The presence and quality of development processes are seen by multiple interviewees as an important marker of a mature and sustainable OSS project.
Release management	The release process should describe the governance and planning of how releases are made and at what cadence.

Security management	The implementation and management of proactive and reactive measures to prevent and address security concerns of the OSS project.
Scaffolding	The availability and quality of the development and communication infrastructure used in the OSS project.
Production output	
Documentation	The presence and quality of documentation for the OSS project considering different stakeholders' perspectives, including developers and end-users.
Technical quality	The technical quality of the OSS and its source code, e.g., in terms of its architecture, source code, and other quality attributes.

4.1 Community Productivity

4.1.1 Social Activity

Concerns about the activity from the OSS project's community and maintainers both in online channels and physically offline (e.g., I5 and I11). Online, the activity can be in the shape of posts, discussions, and interactions in mailing lists, issue trackers, or the community's own social media channels. Offline, the activity may be in the form of dedicated conferences, hackathons, and meetups or contributions to such events but with a more general focus.

Beyond the focal OSS project under investigation, the presence of social communication with upstream and downstream projects was also stressed (I6). For libraries, it is mostly important to have active relationships with downstream users. For end-user-facing projects, relationships with upstream projects are important (I6).

Social activity is seen as a sign of healthy development and a social pulse of the OSS project. It may also provide a proxy for the popularity of a project, which by many interviewees was considered an ambiguous yet essential aspect of a project (e.g., I6 and I12).

The view on social activity varies through the life cycle stages of a project (I11). In the early phases, growth signals are important, e.g., through activity in communication and visibility on public events. A decline phase may be noted when the communication and technical activity slows down, although this can also be sign of a project that is stabilizing why additional aspects should be consulted.

Related work [47] confirms many of the signals raised mentioning outward visibility highlighting discussion activity and retention [11] across social media [18] as well as mailing lists, forums, pull requests, and issues [3, 43]. The activity of the maintainer is also stressed [9, 37]. Offline presence is less stressed, as is the ecosystem activity, although both are mentioned [57, 58].

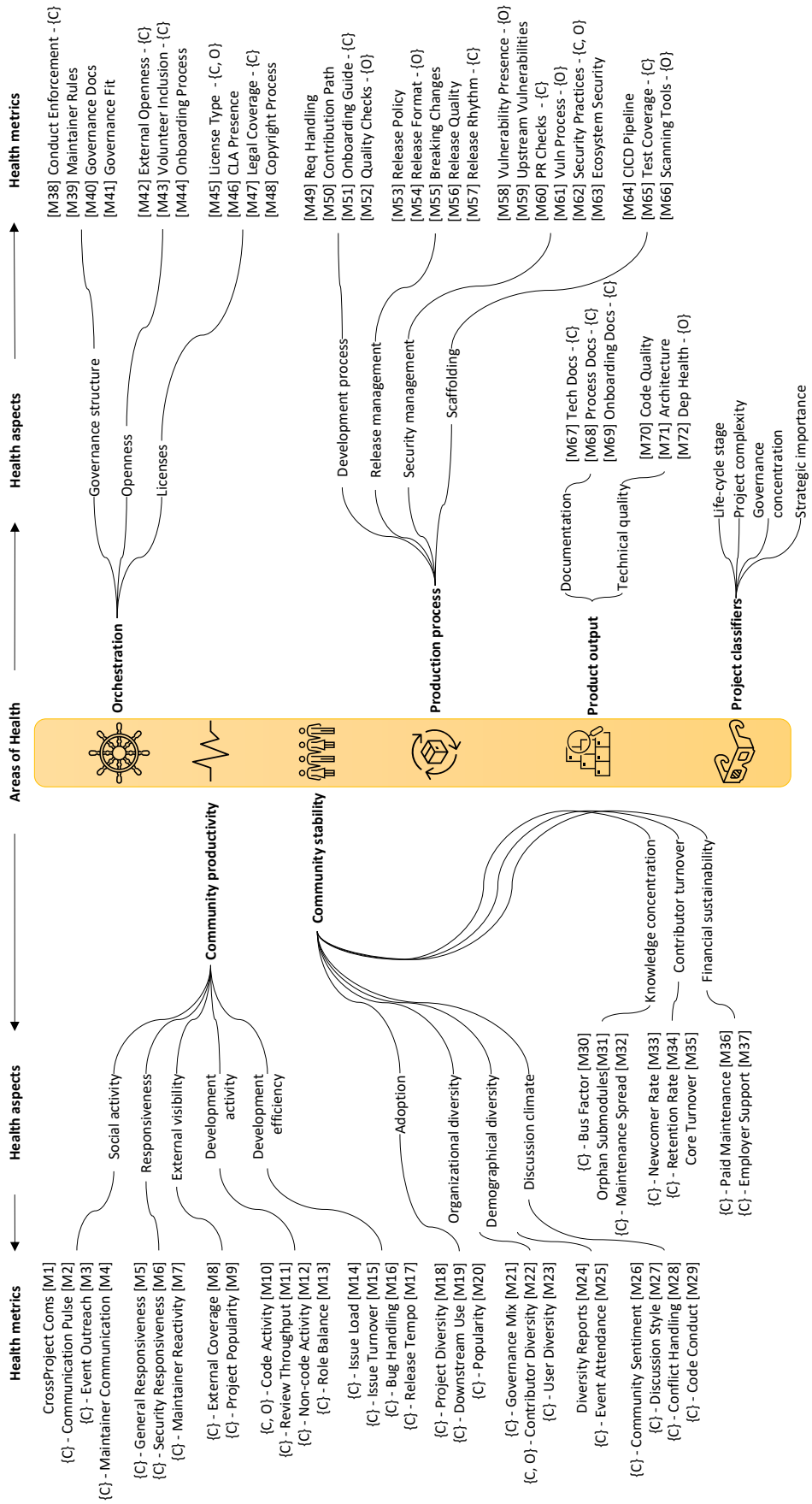

Fig. 2 Overview of the 21 health aspects and related 72 metrics divided across five areas, that help to characterize the health of an OSS project in an organizational intake process. Overlap with the CHAOSS (C) and OpenSSF Scorecard frameworks (O) indicated for each metric.

4.1.2 Responsiveness

Responsiveness concerns the time to respond from the maintainers and community towards, e.g., discussions, pull requests, or issues (e.g., I2 and I8). The type of response depends on the medium and may, e.g., include an answer to a question, a code review of a pull request, or the prioritization or closing of an issue. Security- and vulnerability-related topics and issues were highlighted as necessary in terms of responsiveness due to their critical nature (e.g., I1 and I9). Some see responsiveness as a proxy for the availability of the maintainers, their workload, and the size and activity of the community in general (I2). By extension, the aspect is a sign of a community's productivity. Long lead times can signal bottlenecks in the community (I12), that the project has been abandoned or is in the process of becoming. The aspect may vary depending on the life-cycle stage (I8), yet should still be short, as responsiveness to vulnerabilities is essential regardless of whether a project is in its growth or stability phase. Accordingly, observing the responsiveness over time is important to catch any negative trends early on.

Related work [47] iterate explicit time measures, e.g., response time, time to answer questions, and issue resolution time, as explicit metrics of responsiveness [10, 80]. The quality, correctness and tonality of the answers is also important markers for a healthy and welcoming community [33, 84]. Reactivity from the maintainer is also highlighted as key indicator [9, 37]. Responsiveness to security related issues was not covered explicitly [47].

4.1.3 External Visibility

Visibility of the activity from the individuals engaged in the community to an outside audience. Simple signs of appreciation or attention of individuals, such as stars or followers, as well as reporting on external events, social media, forums, and news, are some potential indicators (I11 and I6).

Increased visibility can be a sign of increased external interest, which in turn can provide a foundation for, e.g., increased attraction of contributors, and chances for securing sponsorships (I6). However, it may not necessarily be a sign of increased adoption or development. Hence, as with many other aspects it needs to be consider in the context of complementary metrics to, e.g., gain an understanding of a project's popularity.

Visibility can be expected to vary across the life cycle of an OSS project (I11) related to a "hype" or new release of a project. Low overall visibility can be a sign of demise, but the project is also feature complete, i.e., stable, which is not necessarily negative (I13). Complementary metrics are hence needed to define where the life cycle of a project resides before considering this aspect in detail.

Related work [47] focus on visibility through concrete measures such as stars, watchers, number of downloads, social media hits, and web page requests used as indicators of this capacity [18, 21]. Such signals is also considered as

popularity indicators, , helping to “instill confidence in new users" and guide choices between similar packages [36, 53, 60, 90].

4.1.4 Development Activity

Concerns the overall development activity by the community, including the many technical aspects and deliverables of the OSS project. Both code and non-code contributions should be considered (e.g., I6 and I16), although the latter may be more difficult to measure (I6). These may, e.g., be directed to the code base, documentation, peer-review and quality assurance process, or release management process.

The development activity is considered one of the more critical aspects by interviewees, signaling that support can potentially be provided if something were to go wrong (I3). I5 and I11 highlight that it should be considered with other aspects, such as social activity, to get an overall pulse of the OSS project. Looking at the activity historically, one can also guess how it may look onward (e.g., I7 and I13).

The development activity should be analyzed both on the project as a whole and in terms of its potentially different submodules or parts. A low or declining activity may be a sign of an unmaintained (orphan) code (I12). I16 further adds the importance of considering the activity in relation to the quality of the work, e.g., in terms of source code and documentation.

Another perspective provided by several interviewees is that the development activity should be put in relation to the number of maintainers and contributors doing the actual work. The ratio, tying into the aspect of knowledge concentration, can give further hints on how the workload distribution looks. Ideally, the project should have activity from both the maintainers and contributors (long-term and episodic) (I16).

Related work [47] treat both non-code [5, 12, 95, 98] and code activity (commits, releases, bug reports, feature requests, patches) [11, 17, 53, 60, 78, 91, 95] as central indicators of technical success, and project activity. The number and activity level of contributors, active contributors, and core team are repeatedly linked to ecosystem or project sustainability [5, 21, 33, 58, 91]. Review throughput is also considered such as effectiveness (closed requests), likelihood and time for task completion, and delay or difficulty in accepting contributions [25, 78, 84].

4.1.5 Development Efficiency

Concerns the effectiveness and ease of the maintainers and contributors of an OSS project in managing and moving the development forward. E.g., in terms of addressing, reviewing, merging, and closing pull requests, as well as performing regular and timely releases.

Development efficiency relates to the responsiveness in the communication of the OSS projects but speaks to the progression and productivity of the

development (I9). Hence, it should be contrasted to the development activity, which describes the pulse of the general development, and less about the quality and productivity of the development (I16).

A growing backlog of unaddressed issues or PRs, or a growing ratio of those addressed, can be a bad sign, raising the question of whether the community can manage the workload (I15). The efficiency should, therefore, also be contrasted against, e.g., the knowledge concentration to get further perspective on the work distribution.

Cadence and consistency in terms of releases and updates is regarded as an important feature of a stable project (e.g., I16 and I17). Timeliness is especially highlighted for security-related issues and solutions, from how they are managed to being released (e.g., I5 and I15), which can signal how the project prioritizes its work.

Related work [47] frames productivity and evolution as continuous growth in content (modules, components, functionality, manuals), efficient transformation of inputs into features, and module or framework growth, capturing a communitiy's ability to expand and evolve over time [56, 66, 67, 95]. Several measures focus on how effectively and quickly projects address community inputs such as issues (bugs, feature requests), and release frequency [25, 56].

4.2 Community Stability

4.2.1 Adoption

Usage and technical adoption of the OSS project as a dependency in downstream software projects and by end-users is considered by several interviewees as an important marker for a healthy project (e.g., I16 and I3). Presence of actors in the community (I11), recorded dependencies (I5), and other popularity indicators (I12) may provide different signs of adoption.

Interviewees highlight that several metrics would be needed to get a fair understanding of the adoption as it is considered difficult to put a number on (I2). Also, several interviewees highlight that the technical adoption will vary pending the life cycle stage of the project (e.g., I11 and I16).

Adoption among larger organizations considered a positive sign in terms of quality and that they may be motivated to act if a vulnerability would be introduced (e.g., I5).

Related work [47] considers downloads as well as stars as adoption proxy indicating end-user interest in an OSS project [53, 95], although noting the latter is imperfect, sometimes manipulable signal and that less popular projects may still be attractive for learning and impact [69].

4.2.2 Organizational Diversity

The diversity of organizations within an OSS community in terms of governance, contribution, and adoption of the underpinning project. Some potential

metrics highlighted are the distribution of governance positions (I1) or the relative size of contributions between actors (I9).

A commonly perceived risk in OSS projects with low organizational diversity in terms of contribution and governance is that the project may depend on the agenda of a limited number of organizations and that influence and contributions from other actors could be improved (e.g., I5 and I6). E.g., explicitly when the project is part of a single-vendor business model or implicitly when the number of maintainers or contributors from one organization is significantly larger than other actors. Two potential implications are a change of license or a drop of support for the project (e.g., I17 and I16).

Related work [47] highlights the importance of considering organizational diversity as reliance on a single company or major contributor increases risk for sustainabiilty [17, 21, 91]. Diversity among target audiences and user types (end-users, advanced end-users, system administrators, developers) further shape who can become contributors and how many potential developers exist, influencing attractiveness, recruitment, and development activity [78, 96].

4.2.3 Demographic Diversity

Demographic diversity looks at the individual level of the maintainer and contributors to an OSS project in terms of gender, race, time zone, language, and cultural aspects (e.g., I9 and I12). It may include both online in the social activity and development of the OSS project (I13) and offline at physical events related to the project (I6).

The importance is raised by many interviewees who consider demographical diversity as crucial for open culture and quality of development due to the different perspectives (e.g., I16). Stability and productivity are also implied by the broader exposure and adoption, and by extension, a large surface area for contributions (I11). One interviewee raised the Fedora OSS project as an example of a highly diverse project in terms of contributions despite mainly being driven by Red Hat (I11).

The aspect is, however, also considered difficult to measure practically online due to the general anonymity of a community (I6) and the sensitive and ethical nature of collecting such information (I9). Measuring diversity at physical events and larger formal organizations (such as foundations) is considered more accessible (I6).

Related work [47] confirms practice in that limits in gender, cultural, and geographical diversity are linked to “community smells" and negative collaboration patterns, and therefore important markers of OSS community health [7, 40, 71].

4.2.4 Discussion Climate

Regards the discussion climate in the community in regard to sentiment, tone, and manner in answers, messages, and general communication within the OSS

project, and how potential conflicts are managed. Examples include rudeness, deliberate misunderstandings, and closing of issues without reason (I1).

Many interviewees consider the presence of toxicity and heated discussions as markers for an unhealthy community that will have a negative impact on both the attraction and retention of contributors (e.g., I5 and I9). The discussion climate should rather be friendly, constructive, and welcoming to create an inclusive environment, increasing the potential attraction and retention rates of contributors. Toxicity needs to be identified upfront and managed proactively (I9), typically by introducing and enforcing a code of conduct and setting up a governance structure that can manage conflicts in a structured way (I8). The character of the discussion climate in a community is considered difficult to measure beyond observing the dialogues taking place and the potential presence of a code of conduct (I17).

Related work [47] also iterates importance of considering the sentiment [24, 54, 89] and presence of any toxic or rude messages, community smells, and different conflict types (task, affective, process, normative) [20, 72, 84]. High community expectancy, conflicting views, political discussions, resistance to change, and unwelcoming or non-actionable reviews fosters a non-sustainable community environment and attractiveness and retention of newcomers [9, 69, 84]. Cross-communication hurdles, use of multiple languages, slang, irony, and idioms create cultural and linguistic barriers, particularly for non-native English speakers, affecting understanding and collaboration across geographically and culturally diverse communities [33].

4.2.5 Knowledge Concentration

Concerns the concentration or distribution of contributions and knowledge to specific individuals or groupings within an OSS project. The aspect is typically described in terms of, e.g., bus or truck factor, meaning the number of people that have to abandon the project for it to go dormant (e.g., I2 and I8). One interviewee suggests looking at the number of individuals making up certain amounts (e.g., 50 or 80 percent) of the contributions to a project (I14). The knowledge concentration can further regard the project as a whole and its parts or modules. One interviewee refers to the latter as an orphan code (I12), i.e., an unmaintained part of a larger OSS project. Concentration of knowledge is also important to capture in terms of who is resolving bug reports or failing builds, tasks not necessarily done by the contributors (I6).

In the inception phase of a project, the knowledge concentration is typically limited to one or a very few individuals but may also remain valid for later phases of a project's life-cycle (e.g., I2 and I8).

A low level of knowledge concentration may indicate a higher risk of maintainer burnout as the burden can grow overwhelming if, e.g., questions, feature requests, and code reviews increase disproportionally to the amount of time available from the maintainer(s) (I2). Financial sustainability for the maintainer becomes a more critical factor for these projects, as well as the quality, e.g., of documentation that may enable others to take over if needed (I1).

One interviewee highlights that there are higher chances that the project may survive if it's in the confinement of a foundation or larger ecosystem (I6). A related factor concerns the amount and diversity of users and downstream projects, which decreases the potential risks related to a low knowledge concentration (I8). The size and complexity of a project and its strategic importance affect how companies value the risk associated with the low knowledge concentration (e.g., I16 and I17).

Related work [47] confirms practice in the concerns of how a low truck factor indicates strong dependence on a few "heroes" and high adoption risk [2, 19, 65]. If most work or critical expertise is concentrated in one or a few members, continuity becomes highly dependent on them, so measures of effort and expertise balance are proposed to detect risky situations [21]. Contribution motivation and collaboration risks becoming limited as an effect when knowledge and expertise needed to deploy and develop software are unevenly distributed [4, 10].

4.2.6 Contributor Turnover

Contributor turnover regards the attraction, retention, and attrition of maintainers and contributors to an OSS project. This also includes episodic volunteers who leave after one or a limited number of contributions.

The turnover is seen by many of the interviewees (e.g., I6 and I9) as a key aspect describing the stability and resilience of an OSS project. Episodic volunteers specifically are seen from different perspectives. On one side, they are considered to add to the load of the maintainer without contributing to the long-term sustainability of the project (I9 and I6). Still, it provides a signal that there is interest in the project and its receptability to contributions (I2).

Retaining the episodic volunteers to stay on as long-term contributors and potentially maintainers is further considered a challenge and a narrow funnel, yet a key sign for an attractive and inclusive community (I9 and I13). On the opposite, if the attrition rate is greater than the attraction and retention, i.e., when there is a negative turnover, companies should be vary of the stability of the concerned project. Turnover can, however, also be a good sign in terms of maintainers as it signals an organic growth in the community, that governance is not locked-in (I13).

Related work [47] considers developer retention as key for project sustainability, with work examining what factors influence newcomers' continued participation and showing that only a minority of contributors become long-term members, while many projects fail to attract or keep a community [46, 49, 55, 85]. Metrics such as growth of developers, new members over time, and repeat contributors capture a community's capacity to replenish its contributor base and avoid decline [17, 21, 29, 92].

4.2.7 Financial Sustainability

Regards the financial situation of maintainers and contributors of OSS projects and whether it enables sustainable and dedicated time for maintenance of the projects. It may concern whether maintainers and contributors are employed, have a personal business set up, or receive sponsorships (I6).

There is a general sense among interviewees that financial sustainability among the maintainers of a project is a positive sign (I5). This implies that they can work professionally with the maintenance while potentially promoting better work-life balance. However, it may also be seen as a risk if the continued maintenance or the attraction of contributions is dependent on, e.g., reoccurring one-time sponsorships (I1). Commercial backing, i.e., when individuals are employed and representing larger organizations, is further seen as a positive sign (I11) but should be seen in contrast to the organization's agendas and the openness for collaboration and contributions if they are in a central position of the project's governance (I2).

Related work [47] note projects' affiliation with organizations and the presence of donors or sponsors provide important financial backing and community characteristics, but sponsorship patterns vary widely and may not always translate into more engineering activity [61, 96]. The ratio between volunteer and paid work is further highlighted as a key indicator of project health [73].

4.3 Orchestration

4.3.1 Governance Structure

Concerns the explicitness, formality, and general recognition of the ecosystem's governance structure and leadership.

Governance and code-of-conduct are considered critical criteria for a healthy OSS project (e.g., I8 and I9) and for proactively managing and minimizing conflicts and toxicity, which by extension, may lead to people abandoning the project. Some interviewees (e.g., I1 and I9) specifically raise the aspect of having rules in place for how maintainership is regulated, distributed, and transitioned when a maintainer leaves a project. Regulations and processes should further be documented and openly available (I5) but continuously revised actively to stay up-to-date (I13).

An OSS project's complexity and stage in its life-cycle heavily impact the requirements and needs for formality in terms of a project's governance (I1). In a small project or the early stages, less rigor is considered acceptable, and concentrated leadership is the focus of development and agility (I8). As the project grows and increases in complexity, more mature governance is preferred by companies that ensure a neutral space for collaboration and provide means of settling disputes and agreeing on a common agenda (e.g., I5 and I11), considered especially important in cases where there are conflicting busi-

ness incentives present (I10). I9 highlights with reference to the Kubernetes project that an efficient governance structure should enable a decentralized development where decisions are taken at a level as low as possible.

Governance is, however, considered difficult to measure quantitatively, although one may look for the presence of specific paragraphs or types of documents, such as a code of conduct (I2 and I8).

Related work [47] highlights importance of enforcing contribution guidelines, maintaining cohesion via connected social networks, and building trustworthiness through stable partnerships and transparent information [10, 21]. Concrete governance setups, such as centralized decision structures, responsibility assignment, support from foundations or firms, and whether projects are managed by larger organizations—are treated as key orchestration mechanisms that balance coherence, participation, and scalability [3, 94].

4.3.2 Openness

Regards to what extent the OSS project is welcoming to and accepting contributions and considering new ideas and general input and influence on the project's development from existing and new contributors (e.g., I1 and I2).

Openness is considered a significant factor for projects deemed strategically essential and where the governance and ownership are centered on one (or a limited number of) organization(s). One interviewee describes how their engineers typically enter a discussion with an OSS project about the ability to influence and the potential implementation of specific use cases (I17).

The openness is also considered crucial for the potential of a vibrant community which is an enabler for many of the benefits that OSS may bring (I16). The opposite imposes a negative culture that can impact the attraction and retention of people in the community.

Related work [47] describes unwillingness to compromise, "can't be wrong" attitudes, and opaque, exclusive PMC communication are described as toxic patterns that make participation hard, push people to drop out, and discourage newcomers, directly undermining community openness [33]. Centralized governance structures can ensure coherence yet also lead to disputes, behind-the-scenes decisions, and questions about accommodating large external contributors, illustrating tensions between hierarchical control and openness to wider participation [3, 94]. Onboarding processes are iterated as a concrete means of improving openness [84].

4.3.3 Licenses

Concerns license-related aspects and processes of managing and distributing the intellectual property maintained by the OSS project.

Many interviewees highlight that a copyleft license may have a negative impact on company participation (e.g., I10 and I13). Factors such as license compatibility and flexibility with different business models are, however, highlighted as impacting factors (I12). The presence of a CLA may also impact

the contribution process by adding friction, inhibiting contributions (I6), and raising the risk of license changes if there is a corporate entity driving the project (I17). However, some interviewees diverge in that it can both be a sign of maturity (e.g., I10 and I17). Export control may be another issue that can restrict or inhibit adoption from companies (I13).

From a commercial user perspective, the trustworthiness of the source of the OSS project in terms of its license compliance and correctness and how it may be verified with data from other sources (e.g., license databases) is specifically highlighted (I1). Depending on the business criticality of the project, a company has different requirements. For less critical projects, trusting the community and potentially performing a manual inspection in the hosting platform may be enough. For more critical OSS used in products, metadata may not be trusted, warranting scans and thorough compliance reviews (I1). Should the project reside within a more professional setting, such as a foundation, trust will probably be higher (I1).

Related work [47] highlights the presence of an explicit OSS license as a key criteria for an OSS project [91]. Type of license in terms of copyleft or permissiveness also impacts how attractive a project may be to commercial actors, where weak copyleft and permissive licenses are preferred by some [91, 94]. The risk of license change is also highlighted as negative for health and attractiveness in cases where governance and ownership is concentrated to one or a few actors [3, 79]

4.4 Production Process

4.4.1 Development Process

The presence and quality of development processes are seen by multiple interviewees as an important marker of a mature and sustainable OSS project (e.g., I12 and I2). Each OSS project may be expected to have different ways of working, so it is important to consider the differences (I2). The development process encompasses multiple parts, including requirements engineering, design, implementation, testing, and release management.

The requirements engineering process should describe how new and existing requirements are managed (I1), e.g., through an issue tracker, how these are discussed and prioritized, and planned for, both in releases and on the more forward looking roadmap.

The development process should further provide guidance on how development is performed, including a contribution process describing how contributions are made and managed (e.g., I2 and I13). A process entailing how newcomers to a project can start to engage in the project is also highlighted among several interviewees (e.g., I1 and I13). The use of modern infrastructure for developing the OSS is a sign of process maturity and up-to-date in terms of modern ways of working (e.g., I1 and I16). Automated testing, CI/CD, and automatic scanning tools are highlighted examples (e.g., I5 and I6).

Related work [47] mentions the need for onboarding and clear contriubution guidelines to help newcomers, e.g., locate suitable tasks [33,69,84]. Active coordination and requirements processes [102], as well as quality checks and processes is also iterated. Presence of unit tests lowers contribution barriers by giving contributors confidence that changes do not break functionality, while speedy issue closure and related metrics are proposed as indicators of team quality and organized support processes [9,36].

4.4.2 Release Management

The release process should describe the governance and planning of how releases are made and at what cadence. Handling security and bug fixes is of extra importance (e.g., I2 and I16). Releases should be clearly described and documented, including impacting dependencies (I14).

Related work [47] view release management as a dynamic capability; higher average release frequency is argued to better satisfy user needs in high-velocity environments and is hypothesized to be positively associated with development sustainability [25,33]. Project releases are further used as a factor when assessing technical success and refinement of the software [53]. Communities' willingness to align with and commit to decided release dates is further viewed as an indicator of readiness to accommodate corporate interests and to coordinate releases with external stakeholders [80].

4.4.3 Security Management

Software security is a pivotal area for all types of software (I11). One interviewee considers it difficult to predict the risk for vulnerabilities and that many vulnerabilities may be a good sign, just like none as a community might be good at identifying new ones (I2). Yet, the number of past and present vulnerabilities and the responsiveness in how these were resolved are highlighted by several interviewees as important numbers to consider (e.g., I10 and I1). Two interviewees note that it is important for users to be aware of their risk appetite (I13) and the level of trust they put into concerned OSS projects (I3).

The dependencies are another important aspect to consider. If a project depends on a lot of other upstream projects, the risk increases for vulnerabilities being imported (I16). The presence of vulnerabilities in dependencies is, therefore, also a potential red flag that should be scanned for (I15). However, this is considered by many interviewees as a difficult and costly procedure, especially in relation to how thorough and far up the dependency tree one wants to go (e.g., I16 and I1). One interviewee notes the importance of observing which ecosystem a project belongs to, as each works differently in terms of how releases and dependencies are managed (I2). In NPM, e.g., one is dependent on each project to update, while in Maven, updates can be brute forced.

Security practices are important for all types of OSS projects (e.g., I17 and I5), including smaller ones, as these can often have central positions in dependency networks, allowing potential vulnerabilities to propagate widely

in software supply chains (I1). One interviewee notes that smaller projects might not have as many security updates as larger projects, implying that they can have different security practices without it being a problem for the health of the project (I16). Regardless, there should always be a strict review and quality assurance process in place for PRs before being merged into the project code base (I10).

Processes should further be in place and transparently documented regarding how vulnerabilities are reported, discussed, managed, disclosed, and communicated (e.g., I1 and I10). Contact details should be openly disclosed for reporting and discussion of security-related information (I14). Adopting best practices as suggested by industry best practice programs such as those of the Open Source Security Foundation is also highlighted (I6), including measures such as enabling multi factor authorization (I4), and running automated security and vulnerability scanning (I16).

Related work [47] frames security as a critical practice for any OSS project [81], highlighting need for dependency-management practices [15, 30, 90], with automated screening tests for both the project [13,15], and dependency-networks [16]. Ongoing maintenance and update capacity are additional key security concerns [3].

4.4.4 Scaffolding

Scaffolding concerns the availability and quality of the development and communication infrastructure used in the OSS project. The presence and use of CI/CD, as well as test automation, were especially highlighted as important markers of technical quality (e.g., I10 and I15). One interviewee describes how it creates trust in the quality assurance practices of the community (I13). The presence of code scanning tools, e.g., for static component analysis and fuzz testing, and the presence of vulnerabilities (e.g., I6 and I16) were highlighted as markers of a more mature project, not to be expected of projects in earlier life-cycle stages.

Related work [47] also iterates the need for infrastructure that enables communication, preserving interaction history, ensuring access to the right mailing lists, and providing accessible communication processes, so contributors do not miss important context and can follow discussions even when offline [33, 80]. Hosting of projects under organizational GitHub accounts and providing continuous-integration infrastructure are further seen as signals of stability [9, 91].

4.5 Production Output

4.5.1 Documentation

Documentation is considered critical among many of the interviewees, both from a developer (I10) and user perspective. It enables the capture, persistence,

and dissemination of knowledge among the existing community as well as newcomers. It is not something that is always prioritized (e.g., I1 and I13) and should cover both deliverables and processes. Deliverables include source code, release notes, and export control (e.g., I15 and I1), while typical processes and process-related artifacts may include onboarding, contribution guidelines, requirements planning and roadmap, training, and FAQs (I13).

General quality attributes of the documentation, including its availability, correctness, and completeness, were further highlighted (e.g., I6 and I3). Accessibility was also raised in terms of enabling the visually impaired, different screen sizes, and mental structure of the text (I6). Such aspects may require a manual inspection to evaluate, although some quantitative metrics may be used, such as its revision history, when measuring development activity.

Related work [47] iterates missing, vague, or outdated documentation as critical signals to look for, including absent contribution instructions, unclear release-process descriptions, undocumented project expectations, and sparse code comments [10, 33, 84]. Clear, complete technical and process documentation are important for easy onboarding of newcomers, including setup instructions, development-environment guides, and structural code documentation [9, 84]. For adopters, well-documented software is explicitly mentioned as a criterion in component selection, underscoring documentation quality as part of perceived project health and robustness [3].

4.5.2 Technical Quality

Concerns the technical quality of the OSS and its source code, e.g., in terms of its architecture, source code, and other quality attributes. Several interviewees highlight readability, clean code, and general adherence to common coding conventions as important markers (e.g., I15 and I17). A user needs to be able to understand the code and be able to build on top of it (I3). The architecture should further be investigated if it makes logical sense (I1) and preferably has a modularized structure that is easy to extend and add to (I6). The presence of circular dependencies, or different versions of the same project/package, is also highlighted as something to look for (I1).

Related work [47] frames high code complexity as common community issues and risk factors for abandonment, because uncontrolled complexity demands more work and information from contributors and can even force costly rewrites when code becomes too hard to change [10, 32, 94]. Undocumented, poorly designed, complex, or buggy codebases are seen as technical hurdles and contribution barriers [9, 33, 69]. Practices such as backporting releases to reduce maintenance costs for dependents, and analyzing code smells and their relation to community smells, are also used as indicators of technical quality and ecosystem sustainability [62, 91]. Inappropriate or eroded design and architecture are also reported as contribution barriers that make projects harder to understand and maintain, reinforcing that architectural quality underpins both contributor experience and long-term project success [9, 94].

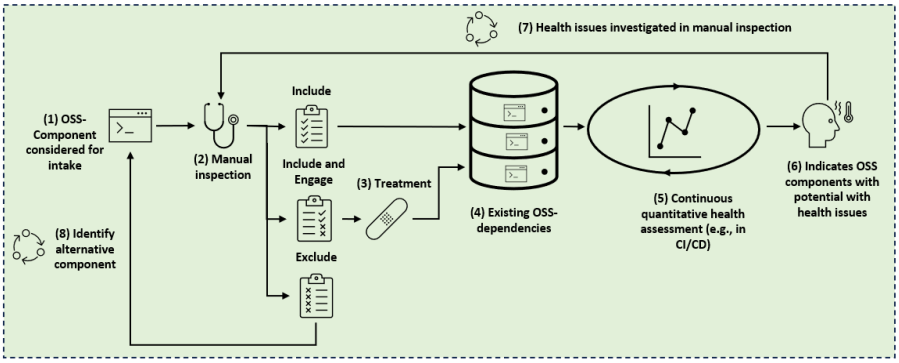

Fig. 3 Overview of the health assessment process, part of an organization's overarching intake process for OSS components.

5 Case study: Implementation at a Case Company

The identified health aspects and related metrics (see Fig. 2) provides a knowledge base for organizations to pick-up and learn from when evaluating an OSS project's health in their intake process. In the fourth research cycle, we further explore how health aspects can be narrowed down further to be applicable for practitioners in an organizational intake process, considering a real-world context through a case study at an international automotive manufacturer. The case company is a partner in this study's overarching research project. A process design and questionnaire for health assessment was designed iteratively based on interviews of domain experts within the organization as part of the qualitative interview survey (I1, I3-4, I14-15), complemented with a follow-up focus group and user observations where the questionnaire was applied and evaluated.

The process, as visualized in Fig. 3, is typically triggered (1) by a developer identifying an OSS component and candidate for inclusion as a dependency in the organization's products and operations. If the component meets initial functional requirements, the developer performs a manual inspection (2) using a standardized questionnaire with questions identified as necessary based on the the concerned health metrics. For the case company, a condensed list of health aspects and metrics (see table 3 in Appendix C) were prioritized through the focus group and user observations.

Tool-support should be developed and leveraged to support the developer is answering the questions and identify any health issues. Existing tooling from the CHAOSS project was discussed as potential candidates to start from [10]. The development of the tool-support should iteratively help to refine the questionnaire. The goal is to keep a manual inspection between 15 and 60 minutes, depending on the complexity of the OSS component. The time aspect was raised as a critical aspect in focus groups and user observations as time is

[10] https://chaoss.community/software/

already limited, and the cost for adding additional burden cannot be too high as it will then be de-prioritized and potentially avoided.

Based on the evaluation, the developer will make a triage decision including the three options:

- If the component is considered healthy, the component is included.
- If the component shows health-issues that are manageable, the component is included but with a treatment designed and implemented to address any issues identified.
- If the component shows health-issues that are not manageable, the component is excluded, and the search will continue for an alternative component (8).

For option number two, the treatment (3) should be designed based on the health issues identified. The health aspects and metrics proposed in this study along with the underpinning literature identified in earlier work [47] can provide input, along with best practice established in industry and the OSS community[11]. Interviewees highlighted that suitable treatments should be designed and documented iteratively and connected to the different questions asked in the health assessment. The definition of suitable treatments is, however, beyond the scope of this study.

For OSS included as dependencies (4) to the organization's products and operations, a quantitative analysis is run continuously (5), e.g., as part of the internal continuous integration, delivery, and deployment pipeline. The analysis should align with the questionnaire but exclude aspects that can only be determined qualitatively. The tool-support developed for the manual inspection should preferably be reused, automated, and integrated into an internal notification system that alerts developers responsible for the component when indicators are passed above-identified thresholds (6).

Interviewees note that the inspection is potentially best performed by individuals with context knowledge and experience of the OSS component as this will help to interpret any health issues indicated by the quantitative analysis. The developer assigned to the manual inspection reiterate the inspection (7) focused on the areas identified by the quantitative analysis and enter the triage stage as described above.

Several interviewees both in the case company and in the general sample highlight that the health of an OSS project cannot be generalized by numbers. Hence, it is important as further raised that developers are continuously trained in how to apply the questionnaire and tool-support for the manual inspection. There must be an understanding of what the different questions mean, why they are asked, and how the answers can be interpreted, both in isolation and in combination.

Participants both from the focus groups and user observations raise the need for knowledge sharing and collective learning on OSS health and what it can imply. Iterative workshops and retrospects where developers can share

[11] E.g., `https://github.com/ossf/scorecard`, `https://chaoss.community/`, `https://standard.publiccode.net/`

their assessments, gain feedback and discuss is proposed as a key part. Standardized training modules and local champions who may provide points of contact are also emphasized.

A third point regards the persisting of health assessments in an organization-wide repository where developers across different units and departments can gain insights into how a project has evolved, both from the eyes of manual inspections and from the continuous monitoring of the quantitative analysis. Such records will also enable follow-up on the OSS project's evolution and help provide an understanding of how certain health issues have emerged and how they potentially should be treated.

6 Discussion

Below, we discuss our findings, first by describing how organizations can go about when implementing health assessments into their intake processes, then how they should consider various traits of projects in these assessments. We then highlight differences and overlaps between our proposed health aspects and metrics and the two industry frameworks, CHAOSS and OpenSSF. Finally, we discuss the limitations of our findings and how we addressed threats to the validity of our work.

6.1 Implementing Health Assessments into the Intake Process

Any implementation will depend on the case at hand, and its business, processes, tools, and culture. What aspects and metrics that are prioritized, how they are analyzed and followed-up on (for example) will deviate and overlap to different extents. The case study presented is intended to demonstrate how an implementation can be done, while also eliciting further empirical knowledge from a real-world problem context in how practitioners may assess the health of OSS projects from an organizational intake perspective. With aforementioned caveats in mind, we provide an abstracted process with specific guidelines to consider based on the research approach we used for our case study (see section 3.3).

First, any implementation process should start by mapping out risks and concerns in the organization's context. Cross-functional groups should be consulted, including product management, security, IT management, legal, and engineers, as all bring different perspectives, as also reflected in the proposed aspects and metrics. Aspects suggested by this study can support the process and ensure that that different considerations are taken into account. **Second**, risks and concerns raised should be openly discussed and contextualized from all perspectives, and mapped against aspects proposed by this study (or relevant frameworks such as CHAOSS and OpenSSF Scorecard). Discussions should avoid focusing on metrics at this point.

Third, identified aspects should be prioritized as not all may be equally important or feasible to consider in an assessment process. Risk appetite needs

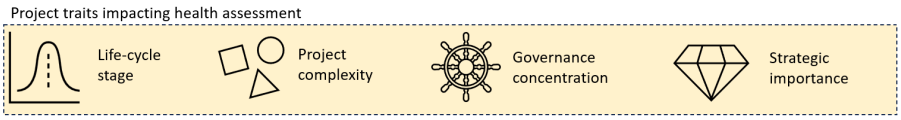

Fig. 4 Four OSS project traits that impact how OSS projects should be assessed and compared in terms of their health.

to be balanced with the time and resources required for health assessment, especially at scale, given the intake process for the full organization concerned. **Fourth**, an investigation needs to evaluate potential metrics aligned with the prioritized aspects. Some metrics can be automated, while others, such as qualitative aspects, may require manual inspection. Tool support may have to be developed or acquired. **Fifth**, a method should be synthesized based on the investigation, considering both manual inspection and tool support, and trialed by engineers and other individuals tasked with assessing the health of OSS components in the intake process.

Finally, the method needs to be operationalized in the intake process, where, again, each organization will have its own practices and processes, which is why no single recipe should be recommended. Inspiration can, however, be drawn from the semi-automated health assessment process identified in the case study, while also considering the key design requirements brought forward. These include the call for a *decentralized* and *self-managed process* where developers *enabled* rather than *overburdened*. Centralized governance and overhead should be kept to a minimum, while the automation and checklists used should be optimized to minimize friction. The process should, however, still be **systematic** in *enabling actionable follow-up* and *insights*.

As the proposed assessment process for the case company highlights (see section 5), an automated analysis based on quantitative metrics should run continuously and forward projects when negative changes are indicated in the relevant metrics. This form for triage is critical as it is not feasible for engineers to manually monitor all dependencies, which can commonly stretch into several thousands for larger organizations. The same automated analysis can further support manual inspection by reducing the time required for single health assessments.

6.2 OSS projects cannot be assessed and compared equally

Our study shows that OSS project health cannot be treated as a single concept, but rather as a set of potential issues comparable to symptoms in living beings, identified through observation and informed questioning. Health assessments should vary by project type, as noted by several interviewees, with traits such as life-cycle stage, complexity, governance concentration, and strategic importance (see Fig. 4) shaping how health aspects and metrics should be applied. These traits influence both the applicability of metrics and the interpreta-

tion of results in an organization's intake process, and need to be considered accordingly. Below, we expand on each of the four traits.

Life-Cycle Stage: Determining a project's life-cycle stage is challenging, as it depends on factors like age, popularity, activity, and maturity, yet defies clear correlation. While prior research has proposed ways to classify and predict OSS evolution [8, 14, 42, 45, 50, 64, 78, 101], interviewees identified four main stages: inception, growth, stabilization, and decline. Inception marks the project's creation, often with concentrated effort from a small team and limited need for governance. Growth follows with technical and social maturation, increasing popularity, and emerging contribution and governance processes as the community expands and diversifies.

Stabilization brings reduced activity and fewer new features but continued maintenance, updates, and emphasis on documentation and openness to handovers. In contrast, decline involves minimal or no activity, lowering quality and relevance, though inactivity can also indicate feature completeness rather than decay. **Future work** should examine how OSS health factors fluctuate across these stages and how priorities and evaluation criteria evolve over time.

Project Complexity: Project complexity encompasses the scope, size, and technical sophistication of an OSS project's codebase, shaping expectations and evaluation criteria. Smaller projects typically prioritize agility over governance, as excessive structure can hinder progress (I11). In contrast, complex projects require mature governance and development processes to manage diverse stakeholder interests and issues such as security (I16). However, even simple projects can be critical, as unnoticed vulnerabilities may propagate widely through software supply chains (I1).

Perceptions of health also vary with complexity: limited maintainership can be acceptable for simple, easily replaceable projects (I16), whereas larger systems like Kubernetes or the Linux kernel are often assumed to be healthy by virtue of their scale (I3C, I16). **Future work** should investigate which dimensions of complexity matter most, both through practitioner perspectives and quantitative analyses of software repositories, as additional complexity factors beyond scope, size, and technical depth may influence OSS health.

Governance Concentration: This category addresses how governance concentration affects an OSS project's openness, transparency, and susceptibility to external influence. Companies view single-entity governance—such as single-vendor OSS models—as risky (I9), fearing restrictive license changes or shifts in technical direction (I10). Contributor License Agreements that transfer copyright to one entity may signal risk (I17), as may high knowledge concentration that leaves the project vulnerable if the company withdraws (I11). Projects hosted by foundations are generally more resilient (I10, I16), though dominance by a few actors can persist (I9, I13). Indicators such as the diversity of contributing companies—sometimes called the elephant factor (I9, I10)—and turnover in leadership roles (I13) can reflect openness and sustainability.

Pay-to-play governance models, where sponsorship buys influence, are red flags (I1), while merit-based governance separating business interests from

technical decisions is preferred (I6). **Future work** should examine how concentrated governance evolves in practice, particularly in single-vendor OSS projects that have shifted toward restrictive or “source-available" licensing. Understanding the real impacts of such changes and when concentrated governance remains acceptable will clarify how existing health metrics apply and where new thresholds are needed.

Strategic Importance: OSS projects deemed critical to an organization’s business carry a lower risk appetite, leading to stricter requirements for health aspects. Business criticality depends on both strategic importance and ease of replacement (I3). Smaller, less strategic projects should have readily available alternatives. For products with long-term maintenance needs, including those with embedded software, OSS components must meet the same standards as internal code (I5, I7). Proactive measures may be necessary to ensure sustainability (I7, I8).

From an actor perspective, financial stability and the reliability of key maintainers, contributors, and hosting foundations are crucial (I2). Greater trust in project members, diverse organizational participation, and openness to external input become especially important in projects with low governance and knowledge concentration (I4, I7). Technically, higher expectations apply to documentation, development processes, and security practices (I3, I4). While strategic importance is widely recognized as influential, how to assess it remains unclear. Future work should explore such evaluation from business and technical perspectives [48], potentially using business model analysis to understand the OSS project’s role in value creation and capture.

6.3 Contrasting to Existing Industry Frameworks

Our mapping of health aspects and metrics identified through our interviews against two industry health frameworks shows both overlap and discrepancies.

Qualitative Aspects missing Support: CHAOSS provides a mature framework that has evolved through a community-driven design process over the past 7 years. Both qualitative and quantitative metrics are available, yet our mapping highlights general limitations in the qualitative areas. The processes and organizational structures raised by interviewees in this study are limited.

Difficult putting a Number on Governance: Governance is one area especially important for organizations looking to take on a potential project, especially one of strategic importance. Ideally, there should be clear and documented rules of procedure for how maintainership is regulated, distributed, and transitioned, e.g., when a maintainer leaves a project (M39), as well as how decisions are made and conflicts managed, by whom, and how they are elected (40). For projects hosted under neutral stewardship, such as non-profit foundations like Eclipse and the Apache Software Foundation, such processes are established, while in independent projects, more or less mature, such details can be briefly or implicitly mentioned in a README file or ingrained in

the tacit knowledge of the community. Defining meaningful quantitative metrics is, therefore, difficult, if not impossible, warranting manual inspections for aspects like this.

Processes often Implicit and Tacit: Similar issues concern processes for requirements engineering (49), contribution guidelines (M50), quality assurance (M52), and release management (M53). Quantitative metrics can assess the occurrence and consistency (e.g., with respect to test coverage and release cadence), while the process details and documentation may remain implicit and known only to the maintainer. While CHAOSS does provide metrics for documentation relating to discoverability[12], usability[13], and accessibility[14], these do not explicitly consider the presence of, e.g., technical (M67), process (M68), and onboarding-related (M69) documentation.

Potential of AI-supported Qualitative Assessments: Defining criteria for qualitative aspects is difficult, as this requires engineers to inspect and develop a common understanding of when something is acceptable. A path for future work, aligning with the continuously expanding AI4SE trend, is to investigate how Large Language Models can be leveraged to automate the process, and at least provide a first indicator of qualitative metrics for specific OSS projects. AI-supported health analysis also has the potential to be integrated into an automated health-assessment and intake process. Quantitative metrics can be fed along with data sources for where qualitative metrics are to be investigated. Proposed assessments should still be evaluated by human beings with contextual knowledge and experience to interpret both the quantitative and qualitative aspects.

No one Framework to Rule Them All: The OpenSSF Scorecard is more limited, with 18 quantitative metrics, all purposefully so to enable full automation of assessments. Its metrics are focused on security aspects, as the interviewees in this study also highlighted as highly important. Here, CHAOSS and OpenSSF Scorecard complement each other, together covering four out of the six security metrics proposed by our interviewees. The complementary nature is also evident in how the OpenSSF Scorecard covers seven metrics that the CHAOSS does not. A recommendation for practice is, hence, not to adopt just one framework, but to consider multiple frameworks in aggregation and tailor them to the needs of the organization at hand, as exemplified by the case study.

6.4 Limitations and threats to validity

To discuss the limitations and threats to validity, we use the criteria defined by Runeson and Höst [75]; *construct validity*, *internal validity*, *external validity*, and *reliability*.

[12] `https://chaoss.community/kb/metric-documentation-discoverability/`
[13] `https://chaoss.community/kb/metric-documentation-usability/`
[14] `https://chaoss.community/kb/metric-documentation-accessibility/`

Construct validity concerns whether what was investigated was actually what the research had in mind. To guide our research, we have taken points from our earlier work and review of OSS health aspects in the literature and used established definitions of health in the planning and execution of our study. Interviewees were introduced to our definition and asked open questions based on our earlier work [47]. In the analysis, transcripts were coded by the two first authors and discussed continuously to reach an agreement. Member checking was also thoroughly performed with all interviewees and study participants, both in cycles 2 and 3 of our research. The member checking included both interview transcripts and copies of the synthesized findings. Only minor comments and input was received, not significantly impacting the definition of health aspects and metrics, or process.

Internal validity considers whether external factors may have influenced the object under investigation. In terms of OSS project health, this may provide a significant threat as OSS health is a very complex construct, as illustrated in our earlier and present work [47]. We, therefore, urge readers to take specific care in the translation and application of our results in a real-world context. Multiple aspects need to be considered in combination together with the risk appetite of the organization. The experience of the individual performing a health assessment will also impact the analysis. Per our case study and the elicited assessment process, we, therefore, recommend that training, peer review, and knowledge sharing be adopted as cornerstones within any organization aiming to establish a health assessment component in their OSS intake process.

External validity considers the generalizability and transferability of the findings to other cases and contexts beyond what has been studied. As we note in Section 6.2, aspects and metrics that are considered relevant and acceptance criteria for these metrics will differ depending on the type of project. Four factors we identified from the interviews include the OSS project's life cycle stage, complexity and governance concentration, as well as strategic importance for the organization doing the health assessment. How these factors (and potential others) impact what aspects to investigate, what acceptance criteria to apply, and in what combinations or order is a comprehensive topic for future research. Until then, we recommend organizations aiming to adopt proposed health aspects and metrics to compare projects of equal complexity, type, life-cycle stage, etc., and to prioritize aspects and metrics similar to the case study presented in Section 5.

Reliability regards the replicability and transparency of the research method applied in the study. We have maintained an audit trail from transcriptions and throughout the coding process. The code book is published in the supplementary material along with the questionnaire and focus group material used in the case study to further enable replicability and transparency in our research process (see Section 7.5).

7 Conclusions

The growing dependence on OSS among organizations increases the need to ensure projects remain healthy—maintained long-term without interruptions or declines in quality. Continuous monitoring and assessment of OSS components, whether newly adopted or in active use, are critical to maintaining robust and reliable software systems. Our study identifies 21 health aspects and 72 metrics (see Appendix A) for use in organizational intake processes, covering five key areas: community productivity and stability, orchestration, production processes, and outputs.

Mapping these aspects against two established industry frameworks (CHAOSS and OpenSSF Scorecard) reveals key gaps in qualitative areas like governance and process documentation, while also showing how the primarily quantitative metrics from both frameworks complement each other. Comparing to extant work [47], supporting evidence is found for 61 out of the 72 metrics. The mapping validates many of the interviewee-proposed metrics in our study and represents a step toward a shared foundation for OSS health assessments within intake processes.

The choice of aspects and metrics will vary depending on each project's context, including industry, market, and technology. Our case study with a large automotive manufacturer demonstrates how the framework can be narrowed and applied in practice. However, interviewees stressed that traits such as lifecycle stage, complexity, governance concentration, and strategic importance should strongly influence how health metrics are selected and interpreted.

Our findings do not detail how these traits map to health aspects, highlighting a topic for future research to address. Practitioners should compare OSS projects only with others of similar traits and prioritize metrics to match internal needs and risk tolerance. By treating OSS health assessments much like medical diagnostics—identifying symptoms early and applying targeted interventions—organizations can reduce risk, ensure long-term viability, and strengthen both their products and the broader OSS ecosystem.

Acknowledgements We would like to thank all of the interviewees for their valuable contribution, which without this research would not be possible.

7.1 Funding

This research was funded through Vinnova grant 2021-02429.

Statements and Declarations

7.2 Ethical approval

Not applicable.

7.3 Informed consent

All interviewees have been provided with informed consent before any data collection activity.

7.4 Author Contributions

All authors have participated in all parts of the research.

7.5 Data Availability Statement

We have made the supplementary material from the empirical investigation for our paper available. It can be accessed at an online repository[15]. Qualitative data is not published per agreement with interviewees to preserve their anonymity.

7.6 Conflict of Interest

The authors declare that they have no conflict of interest.

7.7 Clinical Trial Number

Not applicable.

Appendix A - Metrics per Health Aspect

Below, metrics identified through our qualitative survey with practitioners are listed per health aspect. Beneath each metric, we indicate whether the metric was covered by any of the two industry frameworks CHAOSS [C] or OpenSSF Scorecard [O], and whether any literature from our earlier literature survey [47] provides additional support.

7.7.1 Social Activity

Metrics:

- **M1. CrossProject Coms:** The OSS project's communication with up- and downstream projects, e.g., in reporting and assisting on the discussion of relevant issues (I6).
  - Frameworks: - | Literature: [58]

[15] `https://figshare.com/s/69182a009fb401a89dd2`

- **M2. Communication Pulse:** The OSS project's activity on its communication channels, e.g., mailing lists, issue trackers, or social media.
  - Frameworks: [C] | Literature: [3, 11, 18, 43]
- **M3. Event Outreach:** The OSS project's offline activity through arranging and participating in events, e.g., conferences, hackathons, and meetups.
  - Frameworks: [C] | Literature: [57]
- **M4. Maintainer Communication:** The social activity of the maintainers in contrast to the remaining community on the OSS project's communication channels, e.g., mailing lists, issue trackers, or social media.
  - Frameworks: [C] | Literature: [9, 37]

7.7.2 Responsiveness

Metrics:

- **M5. General Responsiveness:** The timeliness and quality of responses to, e.g., new discussion questions, pull requests, or issues (e.g., I5 and I17).
  - Frameworks: [C] | Literature: [10, 33, 80, 84]
- **M6. Security Responsiveness:** The responsiveness specifically in terms of security-related discussions, pull requests, or issues (e.g., I1 and I10).
  - Frameworks: [C] | Literature:
- **M7. Maintainer Reactivity:** The responsiveness of the maintainer in contrast to the rest of the community (e.g., I1 and I2).
  - Frameworks: [C] | Literature: [9, 37]

7.7.3 External Visibility

Metrics:

- **M8. External Coverage:** The reporting of the project in external events, social media, forums, and news (I11 and I6).
  - Frameworks: [C] | Literature: [18, 21]
- **M9. Project Popularity:** The project's external visibility signaled through various popularity indicators, e.g., stars and followers on GitHub (I12), or downloads from the package manager (I8).
  - Frameworks: [C] | Literature: [36, 53, 60, 90]

7.7.4 Development Activity

Metrics:

- **M10. Code Activity:** The contribution activity to the code base over time, e.g., last 45, 90, and 365 days.
  - Frameworks: [C, O] | Literature: [11, 17, 53, 60, 78, 91, 95]
- **M11. Review Throughput:** The activity in development-related activities, such as code reviews, merging of PRs, and actions on issues over equal periods of time.

  - Frameworks: [C] | Literature: [25, 78, 84]
- **M12. Non-code Activity:** The activity in contributions to non-code tasks, e.g., documentation, test cases, and release management activities over equal periods of time (e.g., I10 and I6).
  - Frameworks: [C] | Literature: [5, 12, 95, 98]
- **M13. Role Balance:** The different types of activities in contrast between the maintainer(s), long-term contributors, and drive-by contributors (I9).
  - Frameworks: [C] | Literature: [5, 21, 33, 58, 91]

7.7.5 Development Efficiency

Metrics:

- **M14. Issue Load:** The size and evolution of the project's backlog in terms of open and unresolved issues (I12).
  - Frameworks: [C] | Literature: [25, 78, 84]
- **M15. Issue Turnover:** The pace in how issues are being addressed and closed and PRs merged (I5).
  - Frameworks: [C] | Literature: [56, 66, 67, 95]
- **M16. Bug Handling:** The corresponding responsiveness towards bugs and security-related issues and PRs (e.g., I5 and I15).
  - Frameworks: [C] | Literature: [25, 78, 84]
- **M17. Release Tempo:** The cadence and timeliness at which releases are made and planned (e.g., I1 and I17).
  - Frameworks: [C] | Literature: [25, 56]

7.8 Community Stability

7.8.1 Adoption

Metrics:

- **M18. Project Diversity:** The diversity of individuals and organizations represented in the project's various communication channels and ongoing development (I11).
  - Frameworks: [C] | Literature:
- **M19. Downstream Use:** The adoption of the project by other downstream projects (I5).
  - Frameworks: [C] | Literature:
- **M20. Popularity:** The project's adoption signaled through various popularity indicators, e.g., stars and followers on GitHub (I12), or downloads from the package manager (I8).
  - Frameworks: [C] | Literature: [53, 95]

7.8.2 Organizational Diversity

Metrics:

- **M21. Governance Mix:** The diversity of organizations represented in the governance of the project and its distribution of positions, e.g., in governance and technical steering committees (I1).
  - Frameworks: [C] | Literature: [17, 21, 91]
- **M22. Contributor Diversity:** The diversity of organizations represented by the individuals contributing to the project and the distribution of contributions among them (I9).
  - Frameworks: [C, O] | Literature: [17, 21, 91]
- **M23. User Diversity:** The diversity of organizations represented among the outspoken users of the project (I5).
  - Frameworks: [C] | Literature: [78, 96]

7.8.3 Demographic Diversity

Metrics:

- **M24. Diversity Reports:** Reports in terms of diversity produced on the project considering, e.g., gender, race, time zone, language, and cultural aspects (I9).
  - Frameworks: | Literature: [7, 40, 71]
- **M25. Event Attendance:** The turnout at physical events arranged by the project? (I6).
  - Frameworks: [C] | Literature:

7.8.4 Discussion Climate

Metrics:

- **M26. Community Sentiment:** The overall sentiment, tone, and manner in answers, messages, and general communication within the OSS project and how it is reflected in the documentation (I1).
  - Frameworks: [C] | Literature: [24, 54, 89]
- **M27. Discussion Style:** Use of slang, irony, or idiomatic expressions in technical discussions.
  - Frameworks: [C] | Literature: [33]
- **M28. Conflict Handling:** Presence of conflicts, and how they are managed (I11).
  - Frameworks: [C] | Literature: [9, 69, 84]
- **M29. Code Conduct:** Presence of a code-of-conduct, and a governance and process for implementing it (I8).
  - Frameworks: [C] | Literature: [84]

7.8.5 Knowledge Concentration

Metrics:

- **M30. Bus Factor:** The number of individuals doing most of the development in the project (i.e., bus factor), e.g., in terms of 50 and 80 percent (I14), and the corresponding number for organizations (i.e., elephant factor).
  - Frameworks: [C] | Literature: [2, 19, 65]
- **M31. Orphan Submodules:** Presence of any submodules or parts in the project with a low bus or elephant factor, also referred to as orphan code (I12).
  - Frameworks: | Literature: [2, 19, 65]
- **M32. Maintenance Spread:** The corresponding bus and elephant factors in terms of who maintenance tasks beyond code contributions, e.g., resolving issues, performing code reviews, or resolving build failures.
  - Frameworks: [C] | Literature: [4, 10]

7.8.6 Contributor Turnover

Metrics:

- **M33. Newcomer Rate:** The number of new contributors attracted to the community over time, e.g., last 45, 90, and 365 days (e.g., I6).
  - Frameworks: [C] | Literature: [17, 21, 29, 92]
- **M34. Retention Rate:** The number of new contributors that have made recurrent contributions and, to some extent, been retained to the community over the same periods.
  - Frameworks: [C] | Literature: [17, 21, 29, 92]
- **M35. Core Turnover:** The turnover of maintainers, long-term contributors, and individuals in central governance positions in the project (I13).
  - Frameworks: | Literature: [46, 49, 55, 85]

7.8.7 Financial Sustainability

Metrics:

- **M36. Paid Maintenance:** The extent maintainers in any way are paid or sponsored to work professionally on maintaining the project, and how (I1).
  - Frameworks: [C] | Literature: [61, 73, 96]
- **M37. Employer Support:** The extent of employed contributors that engage in the project and how financially solid their employers are (e.g., I11 and I16).
  - Frameworks: [C] | Literature: [73]

7.9 Orchestration

7.9.1 Governance Structure

Metrics:

- **M38. Conduct Enforcement:** Presence of a code-of-conduct and governance and process for how it is enforced project (e.g., I8 and I9).
  - Frameworks: [C] | Literature: [10, 21]
- **M39. Maintainer Rules:** Presence of rules for how maintainership is regulated, distributed, and transitioned when a maintainer leaves a project (e.g., I1 and I9).
  - Frameworks: | Literature: [10, 21]
- **M40. Governance Docs:** Presence, availability, and up-to-date documentation of governance in terms of how decisions are made and conflicts managed, by whom, and how they are elected (I5 and I13).
  - Frameworks: | Literature: [10, 21]
- **M41. Governance Fit:** Maturity of the governance of the project compared to its current size and life-cycle stage (I1).
  - Frameworks: | Literature: [3, 94]

7.9.2 Openness

Metrics:

- **M42. External Openness:** The openness for external contributions (I1).
  - Frameworks: [C] | Literature: [3, 33, 94]
- **M43. Volunteer Inclusion:** The extent contributions beyond maintainers and long-term contributors are accepted, e.g., from episodic volunteers (I2).
  - Frameworks: [C] | Literature: [3, 33, 94]
- **M44: Onboarding Process:** Presence of an onboarding process and general support for newcomers for engaging and contributing to the project (I5).
  - Frameworks: | Literature: [84]

7.9.3 Licenses

Metrics:

- **M45. License Type:** The types of OSS licenses the project is published under (e.g., I10 and I13).
  - Frameworks: [C, O] | Literature: [91, 94]
- **M46. CLA Presence:** Presence of a Contributor License Agreement (I6), to whom the copyrights are transferred to (I17).
  - Frameworks: | Literature:
- **M47. Legal Coverage:** Presence and quality of necessary legal information, e.g., in terms of included licenses and export control information (I13).
  - Frameworks: [C] | Literature:
- **M48. Copyright Process:** Presence of a process for managing licenses and copyright within the OSS project.
  - Frameworks: | Literature:

7.10 Production Process

7.10.1 Development Process

Metrics:

- **M49. Req Handling:** Process for how requirements are identified, discussed, prioritized, and planned (I1).
  - Frameworks: | Literature: [102]
- **M50. Contribution Path:** Process describing how contributions should be made and how these are managed (e.g., I2 and I13).
  - Frameworks: | Literature: [33, 69, 84]
- **M51. Onboarding Guide:** Process for onboarding newcomers to the project, e.g., in terms of joining discussions and making contributions (e.g., I1 and I13).
  - Frameworks: [C] | Literature: [33, 69, 84]
- **M52. Quality Checks:** Process and infrastructure for quality assurance of the project, both continuously and per release (I9).
  - Frameworks: [O] | Literature: [9, 36]

7.10.2 Release Management

Metrics:

- **M53. Release Policy:** The structure and transparency of the software release process (e.g., I1 and I14).
  - Frameworks: | Literature: [25, 33]
- **M54. Release Format:** The type of format releases are packaged in and whether they are signed with PGP appropriately (I1).
  - Frameworks: [O] | Literature:
- **M55. Breaking Changes:** Process for managing breaking changes (I2).
  - Frameworks: | Literature:
- **M56. Release Quality:** The quality of releases, e.g., in terms of build quality and documentation (I14).
  - Frameworks: | Literature: [25, 33]
- **M57. Release Rhythm:** The cadence and consistency of releases (e.g., I14 and I17).
  - Frameworks: [C] | Literature: [53, 80]

7.10.3 Security Management

Metrics:

- **M58. Vulnerability Presence :** The presence of vulnerabilities that have been reported, e.g., during the last 1, 3, and 12 months, and at what rate have these been addressed (e.g., I10 and I1).
  - Frameworks: [O] | Literature: [13, 15]

- **M59. Upstream Vulnerabilities:** The corresponding presence in upstream dependencies of the focal OSS project (I15).
  - Frameworks: | Literature: [13, 15]
- **M60. PR Checks:** The review and quality assurance practices for pull requests to the OSS project (I10).
  - Frameworks: [C] | Literature:
- **M61. Vuln Process:** Processes for how vulnerabilities are reported, discussed, managed, disclosed, and communicated by the OSS project (e.g., I1 and I10).
  - Frameworks: [O] | Literature: [15, 30, 90]
- **M62. Security Practices:** Adoption of the best practices suggested by the Open Source Security Foundation (I6), such as enabling multi-factor authorization (I4) and running automated security and vulnerability scanning (I16).
  - Frameworks: [C, O] | Literature: [15, 30, 90]
- **M63. Ecosystem Security:** Security and release management processes in the ecosystem that the focal OSS project resides in (I2).
  - Frameworks: | Literature:

7.10.4 Scaffolding

Metrics:

- **M64. CICD Pipeline:** Presence of a functioning CI/CD pipeline (e.g., I10 and I15), its process, and maintenance of it.
  - Frameworks: | Literature: [9, 91]
- **M65. Test Coverage:** Presence of a functioning test automation (I13), and the test coverage of the project.
  - Frameworks: [C] | Literature: [9, 36]
- **M66. Scanning Tools:** Use of any type of scanning tools, e.g., for static component analysis, dependency updates, fuzz testing, and presence of vulnerabilities (e.g., I6 and I16).
  - Frameworks: [O] | Literature: [13, 15]

7.11 Production Output

7.11.1 Documentation

Metrics:

- **M67. Tech Docs:** The technical documentation of deliverables, including source code and releases, e.g., in terms of completeness, up-to-date, correctness, and accessibility.
  - Frameworks: [C] | Literature: [3]
- **M68. Process Docs:** The process-related documentation, e.g., related to the development process, and governance structure, e.g., in terms of completeness, up-to-date, correctness, and accessibility.

  - Frameworks: [C] | Literature: [10, 33, 84]
- **M69. Onboarding Docs:** The onboarding documentation for enabling newcomers to engage with the OSS project, e.g., in terms of completeness, up-to-date, correctness, and accessibility.
  - Frameworks: [C] | Literature: [9, 84]

7.11.2 Technical Quality

Metrics:

- **M70. Code Quality:** The readability and adherence to common coding conventions of the source code (e.g., I15 and I17).
  - Frameworks: | Literature: [10, 32, 94]
- **M71. Architecture:** The logic and appropriateness of the OSS project's architecture (I1) and whether it has a modular structure (I6).
  - Frameworks: | Literature: [9, 94]
- **M72. Dep Health [O]:** Presence of circular dependencies, or to different versions of the same project/package (I1).
  - Frameworks: [O] | Literature:

Appendix B - Health Assessment Framework from Earlier work

Table 2: High level overview of the 15 themes covering 107 health aspects among the Actor, Software, and Orchestrator perspectives, based on a snowball literature review of 146 papers, as presented in earlier work [47].

Theme	**Description**	# **asp.**
Actors-perspective		
Communication	*"Consider[s] the social interactions internally between the actors within an OSS community, and externally by the community in their outward-facing communication."* [47]	6
Culture	*"Helps to answer the question how able a community is to facilitate a positive and inclusive collaboration and dialogue among existing and potential actors."* [47]	6
Finance	*"'Describes the financial support, and the general financial stability of the actors in the community that are maintaining or contributing to the OSS project.'* [47]	2

Diversity	*"Describes the OSS project's or its overarching ecosystem's ability to be receptive to diversity and self-renew itself."* [47]	8
Popularity	*"Describes the general external interest in the OSS project or its overarching ecosystem."* [47]	8
Stability	*"Describes the resilience and robustness of the OSS community or its overarching ecosystem in terms of their population."* [47]	14
Technical activity	*"Describes the overall technical activity within the OSS project."* [47] 6	
Software-perspective		
Documentation	*"Describes the quality of general and technical documentation."* [47]	6
Development process	*"Describes the quality and formality of the processes and practices for how the development is performed."* [47]	7
License	*"Highlights how license choices and related practices may affect the popularity and attractiveness of an OSS project, both for commercial actors and individuals."* [47]	4
General factors	*"Looks at how attractive an OSS project is based on its general technical features."* [47]	7
Scaffolding	*"Concerns how robust and accessible the development and communication infrastructure used in the OSS project is in terms of enabling a collaborative and high quality maintenance of the project."* [47]	5
Security	*"Helps to answer the question how robust an OSS project is in terms of mitigating and managing vulnerabilities and security-related aspects in the current and future maintenance of the project."* [47]	7
Technical quality	*"Considers both the OSS project in general and its code base specifically."* [47]	12
Orchestration-perspective		
Orchestration	*"Describes the governance structure and quality of the leadership."* [47]	10

Appendix C - Questionnaire for Assessing Health in Case Company

Table 3: Overview of questions and metrics identified and prioritized by the case company to be used for manual inspections in the health assessment as part of the organization's overarching intake process of OSS components. Specific comments included from the focus group participants.

Community productivity - Development activity
A1: The contribution activity to the code base over time, e.g., last 45, 90, and 365 days. A2: The activity in development-related activities, such as code-reviews, merging of PRs, and actions on issues over equal periods of time. General note: *"Important because we need to know if the software is being worked on and is up-to date. Indicates that we will likely have future and current support."*
Community productivity - Responsiveness
A1: The timeliness and quality of responses to, e.g., new discussions questions, pull requests, or issues. General note: The group emphasizes the importance to consider both the social coding platform where a project is hosted, and also external platforms such as StackOverflow where project-related questions may be asked.
Community stability - Adoption
A3: The project's adoption signaled through various popularity indicators, e.g., stars and followers on GitHub, or downloads from the package manager. General note: The group notes that *"More people and good scores means more activity (bug reports/fixes, acceptance, security findings, etc.)"*.
Orchestration - Licenses
A3: Presence and quality of necessary legal information, e.g., in terms of included licenses, and export control information. General note: *"Indicates that legal factors have been taken into consideration"*.
Orchestration - Governance structure
A3: Presence, availability, and up-to-date documentation of governance in terms of how decisions are made, and conflicts managed, by whom, and how they are elected.
Production processes - Security management

A1: The presence of vulnerabilities that has been reported, e.g., during the last 1, 3 and 12 months, and at what rate has these been addressed. General note: *"Shows that we can be confident that bugs are being reported/fixed and builds trust in our usage of it."* A4: Processes for how vulnerabilities are reported, discussed, managed, disclosed, and communicated by the OSS project (e.g., I1 and I10). A5: Adopted of the best practices suggested by the Open Source Security Foundation such as enabling multi factor authorization, and running automated security and vulnerability scanning. General note: *"How do you trust that the Open Source is secure? You can scan the Open Source for security vulnerabilities using, e.g., Dynamic or static analysis".*
Production processes - Development process
A4: Process and infrastructure for quality assurance of the project, both continuously, and per release.
Production processes - Release management
A1: The structure and transparency of the software release process.
Production output - Documentation
A1: The technical documentation of deliverables, including source code and releases, e.g., in terms of completeness, up-to-date, correctness, and accessibility. *"This means that we can utilize the project in the way it is intended to be used. Removes/reduces guess-work as well as less misconfigurations."*
Production output - Technical quality
A1: The readability, and adherence to common coding conventions of the source code. A2: The logic and appropriateness of the OSS project's architecture (I1), and whether it has a modular structure. *"Evaluate 1) against our need and guardrails, 2) readability and understandability, and 3) use of proper structure and coding rules (naming convention, code linting etc.)"*

Appendix D - Interviewee demographics

Table 4: Overview of the interviewees, the sample they belong to (case or general), their title, type of organization they represent, and a brief motivation for their relevance for the study. All interviewees have 10+ years of professional experience in working with OSS from a community or company perspective, either on a technical or strategic level. All interviewees also confirmed that they had repeated experience in analyzing the health of OSS projects, although based on their personal experience and rationale.

ID	**Sample**	**Title**	**Type of Organization**	**Relevance for project**
I1	Case	Open Source Program Officer, Company's OSPO	Automotive manufacturer	Responsible for OSS operations of the case company.
I2	General	Head of Research and Data Science	Data analytics	Responsible for the research and development of product features related to OSS health analytics.
I3	Case	Lead architect within software development tools	Automotive manufacturer	Oversees the sourcing, adoption, and integration of OSS in the area of software development tools.
I4	Case	Senior IT Security Architect	Automotive manufacturer	Responsible for processes and practices related to internal security reviews, including OSS.
I5	General	Expert engineer	Manufacturer of embedded software devices	Internal advocate and expert on OSS development and processes. Maintainer of several OSS projects.
I6	General	Director of Sales	Data analytics	Manages products and services focused on OSS health analytics. Has a PhD. degree on the topic.
I7	General	Open Source Program Officer, Company's OSPO	Car manufacturer	Responsible for the OSS program within the organization.

I8	General	Senior Manager, OSPO	OSS-based service and product provider	Responsible for community outreach and metrics programs, including the development, training, and implementation of Health metrics.
I9	General	Research Analyst, Company's OSPO	Cloud services	Supporting implementation of health metrics internally and engaging in external collaborations on their development.
I10	General	Open Source Manager, Company's OSPO	Telecom infrastructure	Responsible for the introduction of OSS security practices inside the company and externally engaged in OSS Security communities developing best practices on the topic.
I11	General	Consultant	Independent	Former senior manager of OSS operations in large OSS-based service providers.
I12	General	CEO	Data analytics	Manages products and services focused on OSS health analytics. Has a PhD. degree on a related topic.
I13	General	Director of Community and Developer Relations	Cloud-based Database service provider	Engaged in several community initiatives related to OSS health metrics, with leadership experience from OSS foundations.
I14	Case	Senior Solutions Architect	Automotive manufacture	Responsible for the introduction and harmonization of Security practices among development teams.
I15	Case	Security Engineer	Automotive manufacture	Experience in security evaluation and sourcing of OSS components for current and previous employers.
I16	General	Open Source Manager, Company's OSPO	Infrastructure software for cloud services	Responsible for implementing metrics inside the company and engaging in external development. Has a Ph.D. on an adjacent topic.

I17	General	Open Source Program Manager, Company's OSPO	Online Audio platform	Responsible for implementing metrics inside the company and adopting security practices.

References

1. Adewumi, A., Misra, S., Omoregbe, N., Sanz, L.F.: Fosses: Framework for open-source software evaluation and selection. Software: Practice and Experience **49**(5), 780–812 (2019)
2. Avelino, G., Passos, L., Hora, A., Valente, M.T.: A novel approach for estimating truck factors. In: 2016 IEEE 24th Int. conf. on Program Comprehension, pp. 1–10. IEEE (2016)
3. Butler, S., Gamalielsson, J., Lundell, B., Brax, C., Mattsson, A., Gustavsson, T., Feist, J., Kvarnström, B., Lönroth, E.: Considerations and challenges for the adoption of open source components in software-intensive businesses. Journal of Systems and Software **186**, 111152 (2022)
4. Butler, S., Gamalielsson, J., Lundell, B., Brax, C., Sjöberg, J., Mattsson, A., Gustavsson, T., Feist, J., Lönroth, E.: On company contributions to community open source software projects. IEEE Transactions on Software Engineering **47**(7), 1381–1401 (2019)
5. Cánovas Izquierdo, J.L., Cabot, J.: On the analysis of non-coding roles in open source development. Empirical Software Engineering **27**(1), 1–32 (2022)
6. Carvalho, I., Campos, F., Braga, R., David, J.M.N., Stroelle, V., Araújo, M.A.: Heal me-an architecture for health software ecosystem evaluation. In: 2017 IEEE/ACM Joint 5th International Workshop on Software Engineering for Systems-of-Systems and 11th Workshop on Distributed Software Development, Software Ecosystems and Systems-of-Systems (JSOS), pp. 59–65. IEEE (2017)
7. Catolino, G., Palomba, F., Tamburri, D.A., Serebrenik, A., Ferrucci, F.: Gender diversity and women in software teams: How do they affect community smells? In: 2019 IEEE/ACM 41st Int. conf. on Software Engineering: Software Engineering in Society, pp. 11–20. IEEE (2019)
8. Coelho, J., Valente, M.T.: Why modern open source projects fail. In: Proc. of the 2017 11th Joint meeting on Foundations of Software Engineering, pp. 186–196. ACM, New York, NY, USA (2017)
9. Coelho, J., Valente, M.T., Silva, L.L., Hora, A.: Why we engage in floss: Answers from core developers. In: Proceedings of the 11th Int. workshop on Cooperative and Human Aspects of Software Engineering, pp. 114–121. ACM, New York, NY, USA (2018)
10. Constantino, K., Souza, M., Zhou, S., Figueiredo, E., Kästner, C.: Perceptions of open-source software developers on collaborations: An interview and survey study. Journal of Software: Evolution and Process p. e2393 (2021)
11. Constantinou, E., Mens, T.: An empirical comparison of developer retention in the rubygems and npm software ecosystems. Innovations in Systems and Software Engineering **13**(2), 101–115 (2017)
12. Daniel, S., Stewart, K.: Open source project success: Resource access, flow, and integration. The Journal of Strategic Information Systems **25**(3), 159–176 (2016)
13. Dashevskyi, S., Brucker, A.D., Massacci, F.: A screening test for disclosed vulnerabilities in foss components. IEEE Transactions on Software Engineering **45**(10), 945–966 (2018)
14. Decan, A., Constantinou, E., Mens, T., Rocha, H.: Gap: Forecasting commit activity in git projects. Journal of Systems and Software **165**, 110573 (2020)
15. Decan, A., Mens, T., Constantinou, E.: On the evolution of technical lag in the npm package dependency network. In: 2018 IEEE Int. conf. on Software Maintenance and Evolution, pp. 404–414. IEEE (2018)

16. Decan, A., Mens, T., Grosjean, P.: An empirical comparison of dependency network evolution in seven software packaging ecosystems. Empirical Software Engineering **24**(1), 381–416 (2019)
17. Eckert, R., Mueller, A.: Sustainability and diversity of open source software communities. In: The 12th International conf. on Software Engineering Advances, p. 59. IARIA (2017)
18. Fang, H., Lamba, H., Herbsleb, J., Vasilescu, B.: "this is damn slick!" estimating the impact of tweets on open source project popularity and new contributors. In: In 44th Int. conf. on Software Engineering. ACM, New York, NY, USA (2022)
19. Ferreira, M., Valente, M.T., Ferreira, K.: A comparison of three algorithms for computing truck factors. In: 2017 IEEE/ACM 25th Int. conf. on Program Comprehension, pp. 207–217. IEEE (2017)
20. Filippova, A., Cho, H.: The effects and antecedents of conflict in free and open source software development. In: Proc. of the 19th ACM conf. on Computer-Supported Cooperative Work & Social Computing, pp. 705–716. ACM, New York, NY, USA (2016)
21. Franco-Bedoya, O., Ameller, D., Costal, D., Franch, X.: Measuring the quality of open source software ecosystems using queso. In: Int. conf. on Software Technologies, pp. 39–62. Springer, Cham (2014)
22. Franco-Bedoya, O., Ameller, D., Costal, D., Franch, X.: Queso a quality model for open source software ecosystems. In: 2014 9th Int. conf. on Software Engineering and Applications, pp. 209–221. IEEE (2014)
23. Gamalielsson, J., Lundell, B., Lings, B.: The nagios community: An extended quantitative analysis. In: Open Source Software: New Horizons: 6th International IFIP WG 2.13 Conference on Open Source Systems, OSS 2010, Notre Dame, IN, USA, May 30–June 2, 2010. Proceedings 6, pp. 85–96. Springer (2010)
24. Garcia, D., Zanetti, M.S., Schweitzer, F.: The role of emotions in contributors activity: A case study on the gentoo community. In: 2013 Int. conf. on Cloud and Green Computing, pp. 410–417. IEEE (2013)
25. Ghapanchi, A.H.: Predicting software future sustainability: A longitudinal perspective. Information Systems **49**, 40–51 (2015)
26. GitHub: Octoverse: The state of open source and rise of ai in 2023. Tech. rep., GitHub (2023)
27. Goeminne, M., Mens, T.: A framework for analysing and visualising open source software ecosystems. In: Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE), IWPSE-EVOL '10, p. 42–47. Association for Computing Machinery, New York, NY, USA (2010). DOI 10.1145/1862372.1862384
28. Goeminne, M., Mens, T.: Analyzing ecosystems for open source software developer communities. In: Software Ecosystems, pp. 247–275. Edward Elgar Publishing (2013)
29. Goggins, S., Lumbard, K., Germonprez, M.: Open source community health: Analytical metrics and their corresponding narratives. In: 2021 IEEE/ACM 4th Int. Workshop on Software Health in Projects, Ecosystems and Communities, pp. 25–33. ACM, New York, NY, USA (2021)
30. Gonzalez-Barahona, J.M.: Characterizing Outdateness with Technical Lag: An Exploratory Study, p. 735–741. ACM, New York, NY, USA (2020)
31. Gonzalez-Barahona, J.M., Izquierdo-Cortázar, D., Robles, G.: Software development metrics with a purpose. Computer **55**(4), 66–73 (2022). DOI 10.1109/MC.2022.3145680
32. Grigorio, F., Brito, D., Anjos, E., Zenha-Rela, M.: On systems project abandonment: An analysis of complexity during development and evolution of floss systems. In: 2014 IEEE 6th Int. conf. on Adaptive Science & Technology, pp. 1–8. IEEE (2014)
33. Guizani, M., Chatterjee, A., Trinkenreich, B., May, M.E., Noa-Guevara, G.J., Russell, L.J., Cuevas Zambrano, G.G., Izquierdo-Cortazar, D., Steinmacher, I., Gerosa, M.A., Sarma, A.: The long road ahead: Ongoing challenges in contributing to large oss organizations and what to do. Proc. ACM Hum.-Comput. Interact. **5**(CSCW2) (2021). DOI 10.1145/3479551
34. Guizani, M., Zimmermann, T., Sarma, A., Ford, D.: Attracting and retaining oss contributors with a maintainer dashboard. In: In 44th Int. conf. on Software Engineering: Software Engineering in Society. ACM, New York, NY, USA (2022)

35. Hat, R.: The state of enterprise open source. Tech. rep., Red Hat (2022)
36. Jarczyk, O., Gruszka, B., Jaroszewicz, S., Bukowski, L., Wierzbicki, A.: Github projects. quality analysis of open-source software. In: Int. conf. on Social Informatics, pp. 80–94. Springer, Cham (2014)
37. Jiang, S., Cao, J., Prasad, M.: The metrics to evaluate the health status of oss projects based on factor analysis. In: CCF conf. on Computer Supported Cooperative Work and Social Computing, pp. 723–737. Springer, Cham (2019)
38. Kabbedijk, J., Jansen, S.: Steering insight: An exploration of the ruby software ecosystem. In: B. Regnell, I. van de Weerd, O. De Troyer (eds.) Software Business, pp. 44–55. Springer Berlin Heidelberg, Berlin, Heidelberg (2011)
39. Kontio, J., Lehtola, L., Bragge, J.: Using the focus group method in software engineering: obtaining practitioner and user experiences. In: Proceedings. 2004 International Symposium on Empirical Software Engineering, 2004. ISESE'04., pp. 271–280. IEEE (2004)
40. Lambiase, S., Catolino, G., Tamburri, D.A., Serebrenik, A., Palomba, F., Ferrucci, F.: Good fences make good neighbours? on the impact of cultural and geographical dispersion on community smells. In: In 44th Int. conf. on Software Engineering: Software Engineering in Society. ACM, New York, NY, USA (2022)
41. Lazar, J., Feng, J.H., Hochheiser, H.: Research methods in human-computer interaction. Morgan Kaufmann (2017)
42. Li, X., Moreschini, S., Pecorelli, F., Taibi, D.: Ossara: Abandonment risk assessment for embedded open source components. IEEE Software **39**(04), 48–53 (2022)
43. Li, X., Moreschini, S., Zhang, Z., Taibi, D.: Exploring factors and metrics to select open source software components for integration: An empirical study. Journal of Systems and Software **188**, 111255 (2022)
44. Liao, Z., Yi, M., Wang, Y., Liu, S., Liu, H., Zhang, Y., Zhou, Y.: Healthy or not: A way to predict ecosystem health in github. Symmetry **11**(2), 144 (2019)
45. Liao, Z., Zhao, B., Liu, S., Jin, H., He, D., Yang, L., Zhang, Y., Wu, J.: A prediction model of the project life-span in open source software ecosystem. Mobile Networks and Applications **24**(4), 1382–1391 (2019)
46. Lin, B., Robles, G., Serebrenik, A.: Developer turnover in global, industrial open source projects: Insights from applying survival analysis. In: 2017 IEEE 12th Int. conf. on Global Software Engineering, pp. 66–75. IEEE (2017)
47. Linåker, J., Papatheocharous, E., Olsson, T.: How to characterize the health of an open source software project? a snowball literature review of an emerging practice. In: Proceedings of the 18th International Symposium on Open Collaboration, pp. 1–12 (2022)
48. Linåker, J., Regnell, B., Damian, D.: A community strategy framework - how to obtain influence on requirements in meritocratic open source software communities? Information and Software Technology (2019)
49. Link, G.J., Jeske, D.: Understanding organization and open source community relations through the attraction-selection-attrition model. In: Proc. of the 13th Int. Symposium on Open Collaboration, pp. 1–8. Association for Computing Machinery, New York, NY, USA (2017)
50. Macho, H.J., Robles, G.: Preliminary lessons from a software evolution analysis of moodle. In: Proc. of the First Int. conf. on Technological Ecosystem for Enhancing Multiculturality, pp. 157–161. Association for Computing Machinery, New York, NY, USA (2013)
51. Manikas, K., Hansen, K.M.: Reviewing the health of software ecosystems–a conceptual framework proposal. In: Proc. of the 5th Int. Workshop on Software Ecosystems, pp. 33–44. Citeseer (2013)
52. Melegati, J., Conboy, K., Graziotin, D.: Qualitative surveys in software engineering research: Definition, critical review, and guidelines. IEEE Transactions on Software Engineering **50**(12), 3172–3187 (2024). DOI 10.1109/TSE.2024.3474173
53. Midha, V., Palvia, P.: Factors affecting the success of open source software. Journal of Systems and Software **85**(4), 895–905 (2012)
54. Miller, C., Cohen, S., Klug, D., Vasilescu, B., Kästner, C.: "did you miss my comment or what?" understanding toxicity in open source discussions. In: In 44th Int. conf. on Software Engineering, pp. 710–722. ACM, New York, NY, USA (2022)

55. Miller, C., Widder, D.G., Kästner, C., Vasilescu, B.: Why do people give up flossing? a study of contributor disengagement in open source. In: IFIP Int. conf. on Open Source Systems, pp. 116–129. Springer, Cham (2019)
56. Monteith, J.Y., McGregor, J.D., Ingram, J.E.: Proposed metrics on ecosystem health. In: Proc. of the 2014 ACM Int. workshop on Software-defined ecosystems, pp. 33–36. Association for Computing Machinery, New York, NY, USA (2014)
57. Munir, H., Linåker, J., Wnuk, K., Runeson, P., Regnell, B.: Open innovation using open source tools: a case study at sony mobile. Empirical Software Engineering **23**(1), 186–223 (2018)
58. Onyango, S., Steenvoorden, E., Scholten, J., Jansen, S., Gregory, P., Kruchten, P., et al.: Assessing the health of the dark web: An analysis of dark web open source software projects. In: Agile Processes in Software Engineering and Extreme Programming–Workshops, vol. 426, pp. 125–134. Springer (2021)
59. Oriol, M., Müller, C., Marco, J., Fernandez, P., Franch, X., Ruiz-Cortés, A.: Comprehensive assessment of open source software ecosystem health. Internet of Things **22**, 100808 (2023)
60. Osman, K., Baysal, O.: Health is wealth: Evaluating the health of the bitcoin ecosystem in github. In: 2021 IEEE/ACM 4th Int. Workshop on Software Health in Projects, Ecosystems and Communities, pp. 1–8. IEEE (2021)
61. Overney, C., Meinicke, J., Kästner, C., Vasilescu, B.: How to not get rich: An empirical study of donations in open source. In: Proc. of the ACM/IEEE 42nd Int. conf. on Software Engineering, p. 1209–1221. ACM, New York, NY, USA (2020)
62. Palomba, F., Tamburri, D.A.: Predicting the emergence of community smells using socio-technical metrics: A machine-learning approach. Journal of Systems and Software **171**, 110847 (2021)
63. Petrinja, E., Succi, G.: Assessing the open source development processes using omm. Advances in Software Engineering **2012** (2012)
64. Petrinja, E., Succi, G.: Two evolution indicators for foss projects. In: IFIP Int. conf. on Open Source Systems, pp. 216–232. Springer, Cham (2012)
65. Pfeiffer, R.H.: Identifying critical projects via pagerank and truck factor. In: 2021 IEEE/ACM 18th Int. conf. on Mining Software Repositories, pp. 41–45. IEEE (2021)
66. Plakidas, K., Schall, D., Zdun, U.: Evolution of the r software ecosystem: Metrics, relationships, and their impact on qualities. Journal of Systems and Software **132**, 119–146 (2017)
67. Plakidas, K., Stevanetic, S., Schall, D., Ionescu, T.B., Zdun, U.: How do software ecosystems evolve? a quantitative assessment of the r ecosystem. In: Proc. of the 20th Int. Systems and Software Product Line conf., pp. 89–98. Association for Computing Machinery, New York, NY, USA (2016)
68. Poth, A., Levien, D.A., Rrjolli, O., Wanjetscheck, M.: Quality evaluation with the open-source quality-radar for a sustainable selection and use of foss components. In: European Conference on Software Process Improvement, pp. 503–517. Springer (2022)
69. Qiu, H.S., Li, Y.L., Padala, S., Sarma, A., Vasilescu, B.: The signals that potential contributors look for when choosing open-source projects. Proc. ACM Hum.-Comput. Interact. **3**(CSCW) (2019)
70. Qiu, H.S., Lieb, A., Chou, J., Carneal, M., Mok, J., Amspoker, E., Vasilescu, B., Dabbish, L.: Climate coach: A dashboard for open-source maintainers to overview community dynamics. In: Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems, pp. 1–18 (2023)
71. Qiu, H.S., Nolte, A., Brown, A., Serebrenik, A., Vasilescu, B.: Going farther together: The impact of social capital on sustained participation in open source. In: 2019 IEEE/ACM 41st Int. conf. on Software Engineering, pp. 688–699. IEEE (2019)
72. Raman, N., Cao, M., Tsvetkov, Y., Kästner, C., Vasilescu, B.: Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions. In: Proc. of the ACM/IEEE 42nd Int. conf. on Software Engineering: New Ideas and Emerging Results, p. 57–60. ACM, New York, NY, USA (2020)
73. Riehle, D., Riemer, P., Kolassa, C., Schmidt, M.: Paid vs. volunteer work in open source. In: 2014 47th Hawaii Int. conf. on System Sciences, pp. 3286–3295. IEEE (2014)

74. Runeson, P., Höst, M.: Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering **14**(2), 131–164 (2009)
75. Runeson, P., Höst, M., Rainer, A., Regnell, B.: Case Study Research in Software Engineering - Guidelines and Examples. Wiley (2012)
76. Saldaña, J.: The coding manual for qualitative researchers. Sage (2021)
77. Samoladas, I., Gousios, G., Spinellis, D., Stamelos, I.: The sqo-oss quality model: measurement based open source software evaluation. In: Open Source Development, Communities and Quality: IFIP 20 th World Computer Congress, Working Group 2.3 on Open Source Software, September 7-10, 2008, Milano, Italy 4, pp. 237–248. Springer (2008)
78. Santos, C., Kuk, G., Kon, F., Pearson, J.: The attraction of contributors in free and open source software projects. The Journal of Strategic Information Systems **22**(1), 26–45 (2013)
79. Santos, C.D.d.: Changes in free and open source software licenses: managerial interventions and variations on project attractiveness. Journal of Internet Services and Applications **8**(1), 1–12 (2017)
80. Shaikh, M., Levina, N.: Selecting an open innovation community as an alliance partner: Looking for healthy communities and ecosystems. Research Policy **48**(8), 103766 (2019)
81. Silic, M., Back, A.: Open source software adoption: Lessons from linux in munich. IT Professional **19**(1), 42–47 (2017)
82. Singh, J., Gupta, A., Kanwal, P.: The vital role of community in open source software development: A framework for assessment and ranking. Journal of Software: Evolution and Process p. e2643 (2023)
83. Spinellis, D.: How to select open source components. Computer **52**(12), 103–106 (2019)
84. Steinmacher, I., Gerosa, M., Conte, T.U., Redmiles, D.F.: Overcoming social barriers when contributing to open source software projects. Computer Supported Cooperative Work (CSCW) **28**(1), 247–290 (2019)
85. Steinmacher, I., Wiese, I., Chaves, A.P., Gerosa, M.A.: Why do newcomers abandon open source software projects? In: 2013 6th Int. Workshop on Cooperative and Human Aspects of Software Engineering (CHASE), pp. 25–32. IEEE (2013)
86. Stol, K.J., Ali Babar, M.: A comparison framework for open source software evaluation methods. In: Open Source Software: New Horizons: 6th International IFIP WG 2.13 Conference on Open Source Systems, OSS 2010, Notre Dame, IN, USA, May 30–June 2, 2010. Proceedings 6, pp. 389–394. Springer (2010)
87. Synopsis: 2024 open source security and risk analysis report. Tech. rep., Synopsis (2023)
88. Taibi, D., Lavazza, L., Morasca, S.: Openbqr: a framework for the assessment of oss. In: Open Source Development, Adoption and Innovation: IFIP Working Group 2.13 on Open Source Software, June 11–14, 2007, Limerick, Ireland 3, pp. 173–186. Springer (2007)
89. Tourani, P., Jiang, Y., Adams, B.: Monitoring sentiment in open source mailing lists: exploratory study on the apache ecosystem. In: 24th Annual International conf. on Computer Science and Software Engineering, vol. 14, pp. 34–44. IBM (2014)
90. Trockman, A., Zhou, S., Kästner, C., Vasilescu, B.: Adding sparkle to social coding: An empirical study of repository badges in the npm ecosystem. In: Proc. of the 40th Int. conf. on Software Engineering, p. 511–522. ACM, New York, NY, USA (2018)
91. Valiev, M., Vasilescu, B., Herbsleb, J.: Ecosystem-level determinants of sustained activity in open-source projects: A case study of the pypi ecosystem. In: Proc. of the 2018 26th ACM Joint Meeting on European Software Engineering conf. and Symposium on the Foundations of Software Engineering, p. 644–655. ACM, New York, NY, USA (2018)
92. Van Lingen, S., Palomba, A., Lucassen, G.: On the software ecosystem health of open source content management systems. In: 5th Int. workshop on software ecosystems (iwseco 2013), p. 38. Citeseer (2013)
93. Ververs, E., Van Bommel, R., Jansen, S.: Influences on developer participation in the debian software ecosystem. In: Proceedings of the International Conference on Management of Emergent Digital EcoSystems, pp. 89–93 (2011)
94. Viseur, R.: Identifying success factors for the mozilla project. In: IFIP Int. conf. on Open Source Systems, pp. 45–60. Springer, Cham (2013)

95. Vulpen, P.v., Menkveld, A., Jansen, S.: Health measurement of data-scarce software ecosystems: A case study of apple's researchkit. In: Int. conf. of Software Business, pp. 131–145. Springer, Cham (2017)
96. Wang, J.: Survival factors for free open source software projects: A multi-stage perspective. European Management Journal **30**(4), 352–371 (2012)
97. Wang, L., Wan, J., Gao, X.: Toward the health measure for open source software ecosystem via projection pursuit and real-coded accelerated genetic. IEEE Access **7**, 87396–87409 (2019). DOI 10.1109/ACCESS.2019.2926306
98. Wang, Z., Feng, Y., Wang, Y., Jones, J.A., Redmiles, D.: Unveiling elite developers' activities in open source projects. ACM Transactions on Software Engineering and Methodology **29**(3), 1–35 (2020)
99. Wasserman, A., Pal, M., Chan, C.: The business readiness rating model: an evaluation framework for open source. In: Proceedings of the EFOSS Workshop, Como, Italy (2006)
100. Wohlin, C.: Guidelines for snowballing in systematic literature studies and a replication in software engineering. In: Proc. of the 18th Int. conf. on evaluation and assessment in software engineering, pp. 1–10. ACM, New York, NY, USA (2014)
101. Yin, L., Chen, Z., Xuan, Q., Filkov, V.: Sustainability Forecasting for Apache Incubator Projects, p. 1056–1067. ACM, New York, NY, USA (2021)
102. Zhou, S., Vasilescu, B., Kästner, C.: What the fork: A study of inefficient and efficient forking practices in social coding. In: Proc. of the 2019 27th ACM Joint Meeting on European Software Engineering conf. and Symposium on the Foundations of Software Engineering, p. 350–361. ACM, New York, NY, USA (2019)